INTERNSHIP REPORT

on

"Statistical Learning & Implementation"

Submitted by

Rishabh Shukla
BT23MA002
Mathematics and Computing
National Institute of Technology, Mizoram

Internship Conducted at

Indian Institute of Technology, Madras

Under the guidance of
Prof. Neelesh Shankar Upadhye
Department of Mathematics, IIT Madras

Internship Duration:
18 May 2025 – 18 July 2025

Acknowledgement

I am deeply grateful for the opportunity to undertake this internship at the **Indian Institute of Technology Madras**, which significantly enhanced my understanding of statistical learning — both in theory and application.

I would like to express my sincere gratitude to **Prof. S. Sundar**, Director, NIT Mizoram, for referring me to this valuable opportunity and for his continued encouragement throughout my academic journey.

I am especially thankful to my internship supervisor, **Prof. Neelesh Shankar Upadhye**, Department of Mathematics, IIT Madras, for his structured mentorship, insightful guidance, and constructive feedback, which helped me develop strong foundations in both theoretical and practical aspects of machine learning.

I am also grateful to **Dr. Raju Chowdhary**, PhD mentor at IIT Madras, for his constant support in clarifying conceptual doubts and deepening my understanding of complex topics throughout the internship.

I sincerely thank the faculty and staff at **IIT Madras** for providing a collaborative and resource-rich learning environment, and my department — **Mathematics and Computing, NIT Mizoram** — for enabling such academic exposure.

Finally, I appreciate the collaborative spirit and support of my fellow interns, which made this experience all the more enriching.

Abstract

During this eight-week internship, I explored core concepts and methods in **statistical learning**, starting from theoretical foundations to advanced implementation. I began by understanding the role of estimating a function f in supervised learning and the trade-off between **prediction accuracy** and **model interpretability**. I also studied the distinction between **supervised and unsupervised learning**, with a stronger focus on supervised techniques.

I covered essential models such as **linear regression**, **logistic regression**, and k**-Nearest Neighbors**, followed by advanced methods including **ridge**, **lasso**, **elastic net**, **best subset selection**, **stepwise methods**, and **dimension reduction**. Implementing these models from scratch in **Python** deepened my practical understanding.

Later, I explored **non-linear models** like **decision trees** and ensemble methods such as **bagging**, **random forest**, **boosting**, and **XGBoost**. I also studied **Bayesian approaches** to regression using **variational inference (VI)**, **Markov Chain Monte Carlo (MCMC)**, and **Laplace approximation**.

I applied these techniques to datasets like **Boston Housing**, **Heart Disease**, **Credit Card**, and **Diamond Price Prediction**, sourced from **ISLP** and **Kaggle**. These tasks bridged theory and real-world data, strengthening my end-to-end modeling skills.

This report summarizes my learning, implementation, and insights using the books **ISLP** and **PML Volumes 1 and 2** as guides.

Contents

1. Introduction

1.1 About the Internship

This report documents the work completed during my eight-week internship on **Statistical Learning and Implementation**, conducted at the **Indian Institute of Technology Madras** under the guidance of **Prof. Neelesh Shankar Upadhye** from the Department of Mathematics. The internship was designed to develop both theoretical depth and practical proficiency in modern statistical learning methods, combining rigorous study with real-world implementation.

1.2 Motivation and Learning Objectives

The internship began by addressing the fundamental question: how can we estimate an unknown function f that maps input variables X to an output Y in a noisy environment? I explored the importance of this function in supervised learning tasks and studied the key trade-off between **prediction accuracy** and **model interpretability**. Additionally, I learned the differences between **supervised and unsupervised learning**, which laid a strong foundation for the deeper topics covered throughout the internship.

1.3 Topics and Techniques Covered

The internship was divided into multiple phases, each focusing on key areas of statistical learning:

- **Supervised Learning:** I started with classical supervised learning techniques such as **Linear Regression**, **Logistic Regression**, and k**-Nearest Neighbors**. These models formed the foundation of my understanding of regression and classification tasks.
- **Model Selection and Regularization:** I explored **Best Subset Selection**, **Forward and Backward Stepwise Selection**, **Ridge Regression**, **Lasso**, **Elastic Net**, and **Dimension Reduction**.
- **Non-linear Models and Tree-Based Methods:** I studied **Regression Trees**, **Classification Trees**, and **Ensemble Methods** including **Bagging**, **Random Forest**, **Boosting**, and **XGBoost** to handle more complex, non-linear patterns.
- **Probabilistic Modeling and Inference:** In the final phase, I focused on **Bayesian Linear and Logistic Regression** and implemented advanced inference techniques such as **Markov Chain Monte Carlo (MCMC)**, **Variational Inference (VI)**, and **Laplace Approximation**.

1.4 Practical Implementation and Datasets

To reinforce theoretical learning, I implemented almost all algorithms manually from scratch in **Python**, without relying on pre-built model functions. This hands-on approach deepened my conceptual understanding and improved my problem-solving skills.

I also used real-world datasets to test these algorithms, including: **Boston Housing** , **California Housing** , **Heart Disease** , **Hitters** , **Credit Card** , **Carseats** , **Diamond Price Prediction** These datasets, taken from **Kaggle** and the official **ISLP/ISLR resources**.

1.5 Tools and Technologies Used

All implementation was done using the **Python programming language**. For data handling, computation, and visualization, I used essential libraries such as:

- **NumPy** – for numerical computation and array operations
- **Pandas** – for structured data manipulation
- **Matplotlib** and **Seaborn** – for visualization and plotting
- **Scikit-learn** – used only for data preprocessing or comparison; all model logic was manually implemented
- **PyMC3** and **ArviZ** – for Bayesian modeling and inference

The internship emphasized building models without shortcuts, so even core algorithms like **linear regression, logistic regression, k-NN, PCA, Ridge, Lasso, Trees, and Bayesian inference** were coded line-by-line to understand every mathematical detail.

1.6 Model Building and Validation Strategy

Each model developed during the internship was built following a systematic approach:

- I began by splitting the dataset into **training** and **test sets**, typically using an 80-20 or 70-30 split depending on the dataset size.
- For parameter tuning and model selection, I used k**-Fold Cross-Validation**, where the training set is divided into k parts, the model is trained on $k-1$ parts and validated on the remaining one. This process is repeated k times to avoid overfitting and ensure robustness.
- Performance metrics like **Mean Squared Error (MSE)**, **Root Mean Squared Error (RMSE)**, **Accuracy**, $\mathbf{R^2}$, and **Log Loss** were used based on the problem type (regression or classification).
- Model evaluation was done not just by scores but by **residual plots**, **feature importance**, and **interpretability analysis**, especially when using linear models or tree-based methods.

This rigorous validation process ensured that each model generalized well and could be trusted for making reliable predictions.

1.7 Outcomes and Reflection

This internship significantly improved my ability to apply statistical learning techniques to real-world problems. I gained clarity on theoretical principles, experience with implementation challenges, and confidence in debugging and validating models. The experience also taught me how to select the right model for the task, compare alternatives, and interpret results in a statistically sound way.

Overall, this internship bridged the gap between classroom learning and practical application, and gave me a solid base for further research and advanced machine learning work in the future.

2. Statistical Learning

2.1 What is Statistical Learning?

Statistical learning is a way of understanding the relationship between input variables and an output variable using data. In many real-world problems, we cannot directly control the outcome or response variable, but we can control the inputs. For example, a company may not be able to directly increase product sales, but it can adjust advertising spending across different media channels. If we can find a clear relationship between advertising and sales, we can build a model that helps predict future sales and guide decision-making.

In simple terms, statistical learning helps estimate an unknown function f that connects input variables $X_1, X_2, ..., X_p$ to an output variable Y. We assume that this relationship can be written as:

$$Y = f(X) + \varepsilon \tag{2.1}$$

Here, f is the true but unknown function we are trying to estimate, and ε is the error term — it captures all other random factors that affect Y but are not in X. The main goal of statistical learning is to find a good estimate of f using data, so that we can make accurate predictions or better understand how Y depends on X.

Depending on the goal, statistical learning methods can be used for **prediction** or **inference**. When we want to make accurate future predictions, we care more about how close our predicted Y is to the true value. But if our focus is on understanding which input variables influence the output and how, then we care about the interpretability of the model.

In this internship, I explored many methods to estimate f, understand the role of error, and study how well a model generalizes to new data using real data sets. Statistical learning forms the foundation for all the machine learning techniques with which I worked.

2.1.1 Why Estimate f ?

In statistical learning, our main goal is to understand the relationship between the **input variables** (X) and the **output variable** (Y). This relationship is usually represented as:

$$Y = f(X) + \varepsilon \tag{2.1}$$

Here, f is the true but unknown function that links X to Y, and ε is the **random error** that we cannot control. We estimate f because we want to either **predict** future outcomes or **understand** how the inputs affect the output. These two main purposes are known as **prediction** and **inference**.

Prediction:

In many real-world situations, we have access to the input values (X), but the output (Y) is expensive, difficult, or impossible to measure. So we build a model $\hat{f}$ that gives us a **predicted output** $\hat{Y}$ using:

$$\hat{Y} = \hat{f}(X) \tag{2.2}$$

In this setting, we care more about how accurate the prediction is rather than how the model works internally. For example, predicting a patient's risk from a blood test — we just want the prediction to be right. This makes $\hat{f}$ a kind of **black box**.

There are two kinds of errors in prediction:

- **Reducible Error** – comes from inaccuracies in $\hat{f}$; we can reduce this by choosing better models or training methods. - **Irreducible Error** – comes from random noise ($\boldsymbol{\varepsilon}$) or hidden factors we cannot observe or measure; we cannot remove this error.

So, even with the best estimate of $\boldsymbol{f}$, our predictions will always have some **uncertainty**.

Inference:

In other cases, our goal is not just to predict $\boldsymbol{Y}$, but to understand how each input $\boldsymbol{X_1, X_2, ..., X_p}$ affects it. This is called **inference**. Here, we cannot treat the model as a black box — we need to know the exact form of $\hat{f}$.

Inference helps us answer questions like:

- Which **predictors** are important for the response? - Does a **positive or negative relationship** exist between $\boldsymbol{X}$ and $\boldsymbol{Y}$? - Can this relationship be captured by a **simple linear model**, or is it more complex?

For example, if we want to know how much increase in TV advertising leads to a rise in sales, we are doing **inference**, not just prediction.

Sometimes, we care about both — for example, predicting house prices while also knowing which features like location or size influence the price most. In such cases, we aim for a **balance between prediction and interpretability**.

Understanding the purpose — whether it's **prediction**, **inference**, or both — helps in choosing the right modeling approach during the learning process.

2.1.2 How Do We Estimate f?

In statistical learning, our main task is to find a good estimate of the true function $\boldsymbol{f}$ that connects input variables ($\boldsymbol{X}$) with the output ($\boldsymbol{Y}$). To do this, we use the available data — usually called the **training data** — and apply a method that can learn from it. The goal is to build a function $\hat{f}$ such that the predicted value $\hat{\boldsymbol{Y}} = \hat{\boldsymbol{f}}(\boldsymbol{X})$ is as close as possible to the actual $\boldsymbol{Y}$.

There are mainly two types of approaches to estimate $\boldsymbol{f}$: **parametric methods** and **non-parametric methods**.

Parametric Methods

In a parametric approach, we first assume a specific form or structure for the function $\boldsymbol{f}$. A common example is assuming that $\boldsymbol{f}$ is a linear function of the inputs. That means:

$$f(X) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p \tag{2.3}$$

This assumption makes the problem simpler, because now we just need to estimate the parameters $\boldsymbol{\beta_0, \beta_1, \ldots, \beta_p}$. We usually do this using methods like **least squares**. Once the parameters are estimated, we get our final function $\hat{f}$.

The main advantage of parametric methods is that they are easier to compute and require less data. But the disadvantage is that our assumption may be wrong. If the true f is not linear, our model might give bad results. Even if we try to use more flexible forms, it still involves choosing a structure in advance.

Non-Parametric Methods

Non-parametric methods do not assume any fixed form for f. Instead, they try to fit the data as closely as possible without overcomplicating the model. These methods are more flexible and can adapt to many shapes of the true f.

An example is using **splines** to fit a smooth surface to the data points. Non-parametric methods can work really well if we have a lot of data, because they need more information to get a reliable fit.

However, the downside is that they can easily **overfit** — meaning they may follow the training data too closely and perform poorly on new data. That's why we often need to balance between smoothness and accuracy when using non-parametric methods.

To summarize, **parametric methods** are simple and easier to use but may not capture complex patterns in the data, while **non-parametric methods** are more flexible but require more data and care to avoid overfitting. In this internship, I studied both types of methods and understood how and when to use each of them depending on the problem and dataset.

2.1.3 The Trade-Off Between Prediction Accuracy and Model Interpretability

In statistical learning, there is often a trade-off between how well a model can predict and how easy it is to interpret. Some models are **less flexible** — like linear regression — and can only learn simple relationships. But they are very **interpretable**, which means we can clearly understand how each input affects the output.

Other models are **more flexible**, like splines, boosting, or deep learning. These can fit more complex patterns and usually give better predictions. But they are **harder to interpret**, especially when we want to know which variables are actually influencing the result.

If our goal is **inference** — understanding the relationship between X and Y — then it's better to use simpler models like **linear regression**, **subset selection**, or **lasso**. These methods make it easier to explain results. For example, lasso can set some coefficients to exactly zero, helping us identify the most important predictors.

If our goal is **prediction**, we might use more flexible models like **bagging**, **boosting**, or **support vector machines**. But even then, more flexibility doesn't always mean better accuracy. Too much flexibility can lead to **overfitting**, where the model performs well on training data but poorly on new data.

The figure below shows how different methods compare in terms of flexibility and interpretability.

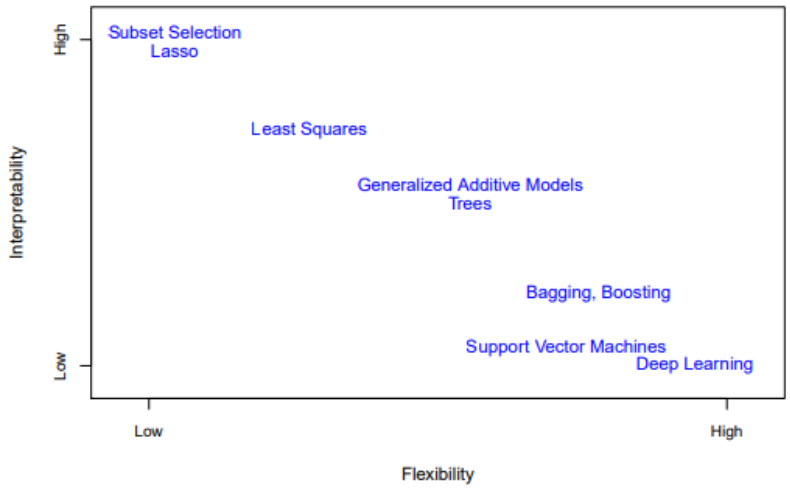


Figure 1: *Trade-off between model flexibility and interpretability.*

In this internship, I learned how to choose the right balance based on the task. When interpretability was more important, I used simple linear models. When prediction was the goal, I explored more flexible models — while being careful about overfitting.

2.1.4 Supervised vs Unsupervised Learning

Most statistical learning problems fall into two main categories: **supervised learning** and **unsupervised learning**.

In **supervised learning**, we have both input variables ($\boldsymbol{X}$) and a known output variable ($\boldsymbol{Y}$). The goal is to learn a function $\boldsymbol{f}$ that connects $\boldsymbol{X}$ to $\boldsymbol{Y}$ so that we can either **predict** $\boldsymbol{Y}$ for new inputs or **understand** the relationship between them. This is the most common setting in machine learning. Examples include **linear regression**, **logistic regression**, **support vector machines**, and **boosting** — all of which I studied and implemented during this internship.

On the other hand, **unsupervised learning** deals with situations where we only have input data ($\boldsymbol{X}$) but no output labels ($\boldsymbol{Y}$). Since there's no response variable to guide the learning, the aim here is to discover **patterns or structure** in the data — such as grouping similar observations together. One common technique for this is **clustering**.

For example, in a market segmentation problem, we may have customer details like income, spending habits, and location, but we don't know who is a "big spender" or "low spender." Using clustering, we can group customers based on similarity in behavior — even without labeled output.

The key difference is: - In **supervised learning**, the model is guided by known outcomes. - In **unsupervised learning**, we explore the data without labels to find hidden patterns.

There's also a third type called **semi-supervised learning**, where we have some observations with known outputs and some without. This happens when collecting outputs is expensive. Though I didn't explore it deeply in this internship, it's useful in many practical scenarios.

2.1.5 Regression vs Classification Problems

In statistical learning, the type of problem we're solving often depends on the kind of response variable ($\boldsymbol{Y}$) we are trying to predict. Based on this, we mainly deal with two types of problems: **regression** and **classification**.

A **regression problem** occurs when the response variable is **quantitative**, meaning it takes numerical values. Examples include predicting a person's income, house prices, or stock market values. In these cases, we aim to estimate a continuous output. A common method used here is **linear regression**.

On the other hand, a **classification problem** arises when the response is **qualitative** or **categorical** — meaning it belongs to one of several categories or classes. For example, predicting whether a person will default on a loan (yes/no), or identifying the type of cancer a patient has based on test results. A popular method for binary classification is **logistic regression**.

Although logistic regression has "regression" in its name, it is actually a classification technique because it predicts the **probability of class membership**. Similarly, methods like **K-nearest neighbors** and **boosting** can be used for both regression and classification, depending on the type of output variable.

In summary: - Use **regression** when predicting **numeric outcomes**. - Use **classification** when predicting **categories or labels**.

2.2 Assessing Model Accuracy

When building any statistical model, it's important to check how well the model actually performs. There is no single method that works best for every dataset — a model that gives great results on one dataset may not perform well on another. That's why assessing model accuracy is a crucial step in the statistical learning process.

Instead of blindly choosing a method, we need to test and compare different models to see which one gives the best performance for the problem we are solving. This helps us pick the right model based on actual results, not just theory.

2.2.1 Measuring the Quality of Fit

To check how good a model is, we need a way to measure how close the model's predictions are to the actual values. The most common way to do this in regression problems is using **Mean Squared Error (MSE)**, which calculates the average of the squared differences between predicted and true values:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2 \tag{2.4}$$

If MSE is small, the model's predictions are close to the actual values. But we must be careful — if we compute MSE on the same data we used to train the model, we get the **training MSE**, which often looks very good. However, what really matters is how well the model performs on new, unseen data — that's measured by the **test MSE**.

In real-world situations, we care more about how well the model performs on future data, not how perfectly it fits the training set. For example, in a medical or financial application, we want the model to give good predictions for new patients or future prices — not just data it has already seen.

One important thing to understand is that **training MSE always decreases** as we make the model more flexible, but **test MSE does not**. If a model becomes too complex, it may start to capture random noise in the training data — this is called **overfitting**. When a model overfits, it performs very well on training data but poorly on new data.

This idea is shown clearly when plotting training and test MSE against model flexibility: training MSE goes down steadily, but test MSE forms a U-shaped curve — first decreasing, then increasing after a point.

The goal is to find the right balance — a model that is flexible enough to capture real patterns but not so flexible that it starts overfitting. In practice, methods like **cross-validation** help estimate test MSE even when a separate test dataset is not available.

During this internship, I understood that blindly minimizing training error is not enough — evaluating model performance on unseen data is essential to ensure generalization.

2.2.2 The Bias–Variance Trade-off

When we build a model, our goal is to make accurate predictions on new data. One of the most important ideas that helps us understand how well a model performs is the **bias–variance trade-off**.

The **expected test error** of a model can be broken into three parts:

- **Bias:** Error caused by simplifying assumptions in the model.
- **Variance:** Error due to the model being too sensitive to small changes in the training data.
- **Irreducible error:** Random noise in the data that cannot be avoided.

Mathematically, it can be written as:

$$E[(y_0 - \hat{f}(x_0))^2] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error} \tag{2.5}$$

If a model is too simple (like linear regression), it may not capture the true pattern in the data — this leads to **high bias**. If a model is too complex and tries to fit every training point, it may perform badly on new data — this leads to **high variance**. A good model needs to balance both.

More flexible models usually have **low bias** but **high variance**. **Less flexible models** tend to have **low variance** but **high bias**. As model flexibility increases, test error initially decreases because bias is reduced, but after a point, the increase in variance dominates and the test error starts increasing — this creates a U-shape curve in test MSE.

The key idea is to find the **right level of flexibility** where both bias and variance are low enough to minimize test error. This trade-off is one of the most important themes in machine learning and statistical modeling.

During my internship, I saw this clearly when comparing simple models like linear regression with more complex ones like splines or ensemble methods. Each model had its own behavior in terms of bias and variance depending on the dataset, and selecting the best model required understanding this balance.

3. Linear Regression

Linear regression is one of the simplest and most widely used techniques in supervised learning. It is mainly used when we want to predict a **quantitative output** based on one or more input variables. Even though it's a basic method, it helps us understand a lot about how variables are related.

From what I've studied, linear regression is a great starting point because it builds the foundation for many advanced machine learning models. The main idea is to fit a straight line that best describes the relationship between input variables and the response variable.

This chapter helped me answer practical questions like:

- Is there a relationship between inputs and output?
- How strong is the relationship?
- Which variables are more useful?
- Can we use this model to make future predictions?

So even though it is simple, linear regression is very powerful when the relationship between variables is roughly linear. It also gives us clear interpretations, which makes it easy to explain the results.

3.1 Simple and Multiple Linear Regression

Simple Linear Regression : is a technique used to model the linear relationship between a **single input (predictor) variable** and a **quantitative output (response) variable**. The assumption is that the output $\boldsymbol{Y}$ changes linearly with respect to the input $\boldsymbol{X}$.

The model can be written as:

$$\boldsymbol{Y = \beta_0 + \beta_1 X + \varepsilon} \tag{3.1}$$

Here, $\boldsymbol{\beta_0}$ is the intercept, $\boldsymbol{\beta_1}$ is the slope (or coefficient) of the predictor variable $\boldsymbol{X}$, and $\boldsymbol{\varepsilon}$ is the error term that captures the noise or variation not explained by the model. The goal is to estimate $\boldsymbol{\beta_0}$ and $\boldsymbol{\beta_1}$ using available data so that the model can make accurate predictions or interpret the effect of $\boldsymbol{X}$ on $\boldsymbol{Y}$.

Multiple Linear Regression : extends the simple model by allowing more than one predictor variable. This is useful when the output $\boldsymbol{Y}$ depends on several inputs, and we want to understand the influence of each predictor while controlling for others.

The model is written as:

$$\boldsymbol{Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon} \tag{3.2}$$

In this case, $\boldsymbol{X_1, X_2, \ldots, X_p}$ are the predictor variables, $\boldsymbol{\beta_1, \beta_2, \ldots, \beta_p}$ are their corresponding coefficients, and $\boldsymbol{\beta_0}$ is the intercept. The objective remains the same — estimate the parameters to understand how each input affects the output and to make accurate predictions.

Both simple and multiple linear regression are foundational tools in statistical modeling because they offer interpretable results and serve as a stepping stone to more advanced models.

3.1.1 Estimating the Coefficients

In linear regression, the goal is to find the best-fit line (in simple regression) or best-fit hyperplane (in multiple regression) that minimizes the difference between the predicted and actual values of the response variable. This "difference" is quantified using a loss function called the **Residual Sum of Squares (RSS)**.

To derive the best coefficients, we take a matrix-based approach, which is more compact and works for both simple and multiple regression.

Let:

- $\boldsymbol{Y}$ be the $\boldsymbol{n \times 1}$ vector of observed output values.
- $\boldsymbol{X}$ be the $\boldsymbol{n \times (p+1)}$ matrix of input features, where the first column is all ones (for the intercept), and the remaining columns represent $\boldsymbol{p}$ predictors.
- $\boldsymbol{\beta}$ be the $\boldsymbol{(p+1) \times 1}$ vector of coefficients, including the intercept.
- $\boldsymbol{\varepsilon}$ be the $\boldsymbol{n \times 1}$ error term.

Then the multiple linear regression model is written in matrix form as:

$$\boldsymbol{Y = X\beta + \varepsilon} \tag{3.3}$$

Objective: We want to choose $\boldsymbol{\beta}$ such that the predicted values $\boldsymbol{\hat{Y} = X\hat{\beta}}$ are as close as possible to the actual $\boldsymbol{Y}$. The closeness is measured using the Residual Sum of Squares (RSS):

$$\text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \boldsymbol{(Y - X\beta)^T (Y - X\beta)} \tag{3.4}$$

Why RSS? RSS tells us how much error our model makes in total. A smaller RSS means our model is fitting the data better. So our goal becomes:

Minimize RSS with respect to $\boldsymbol{\beta}$

Step-by-Step Derivation of the Normal Equation:

To find the value of $\hat{\boldsymbol{\beta}}$ that minimizes RSS, we differentiate the RSS with respect to $\boldsymbol{\beta}$ and set it to zero.

Let's expand the RSS expression:

$$\begin{aligned} \text{RSS} &= \boldsymbol{(Y - X\beta)^T (Y - X\beta)} \\ &= \boldsymbol{Y^T Y - 2\beta^T X^T Y + \beta^T X^T X\beta} \end{aligned}$$

Now take the derivative with respect to $\boldsymbol{\beta}$:

$$\frac{\partial \text{RSS}}{\partial \boldsymbol{\beta}} = \boldsymbol{-2X^T Y + 2X^T X\beta} \tag{3.5}$$

Setting the derivative to zero:

$$-2X^TY + 2X^TX\beta = 0$$
$$X^TX\beta = X^TY$$

This equation is called the **Normal Equation**. Solving it gives the least squares estimate of β:

$$\boxed{\hat{\beta} = (X^TX)^{-1}X^TY} \tag{3.6}$$

Important Note: The matrix X^TX must be invertible. This usually requires that the predictors are linearly independent (i.e., no perfect multicollinearity).

Using This in Simple and Multiple Regression:

In Simple Linear Regression (only one predictor), the X matrix has just two columns: a column of ones and a column of x_i values. The resulting $\hat{\beta}$ is a 2-dimensional vector — one for intercept and one for slope:

$$\hat{\beta}_0 = \text{intercept}, \quad \hat{\beta}_1 = \text{slope}$$

In Multiple Linear Regression, X contains more columns (each predictor becomes a column). The formula still holds:

$$\hat{\beta} = (X^TX)^{-1}X^TY$$

and gives the best coefficients for all predictors simultaneously.

This matrix approach not only simplifies computation, but also provides a unified way to solve both simple and complex regression models.

Once we compute $\hat{\beta}$, we can use it to make predictions using:

$$\hat{Y} = X\hat{\beta}$$

These predictions can then be evaluated using metrics like MSE, RMSE, or R^2, depending on the dataset and context.

3.1.2 Assessing the Accuracy of Coefficient Estimates

Once the regression coefficients $\hat{\beta}$ are estimated, it's important to check how accurate and reliable these estimates are. This helps us understand which predictors are truly useful and how confident we can be about our fitted model.

This assessment involves the following key concepts:

1. Residual Standard Error (RSE):

The **Residual Standard Error** gives an estimate of the average size of the residuals — that is, how far the predictions are from the actual observed values. It's calculated as:

$$\text{RSE} = \sqrt{\frac{\text{RSS}}{n-p-1}} = \sqrt{\frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{n-p-1}} \tag{3.7}$$

Where:

- n: number of observations
- p: number of predictors
- $\hat{y}_i$: predicted value for the i-th observation

Smaller RSE means the model fits the data better.

2. Standard Error of Coefficients:

The **Standard Error (SE)** of a coefficient tells us how much the estimate might vary if we collected new samples from the population.

$$SE(\hat{\beta}_j) = \sqrt{\hat{\sigma}^2 \cdot [(X^T X)^{-1}]_{jj}}, \quad \text{where } \hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1} \tag{3.8}$$

In simple linear regression:

$$SE(\hat{\beta}_1) = \frac{RSE}{\sqrt{\sum (x_i - \bar{x})^2}}, \quad SE(\hat{\beta}_0) = RSE \cdot \sqrt{\frac{1}{n} + \frac{\bar{x}^2}{\sum (x_i - \bar{x})^2}}$$

3. Confidence Interval for Coefficients (95%):

A **95% Confidence Interval** gives a range where the true coefficient likely lies. It's computed as:

$$\hat{\beta}_j \pm t^* \cdot SE(\hat{\beta}_j) \tag{3.9}$$

Where: - t^*: critical value from the t-distribution with $n - p - 1$ degrees of freedom

If the confidence interval contains 0, we are less confident that the predictor significantly affects the response.

4. Hypothesis Testing for Individual Coefficients:

We test whether each predictor variable contributes meaningfully to the model.

$$H_0 : \beta_j = 0 \quad \text{vs} \quad H_1 : \beta_j \neq 0$$

Use the t-statistic:

$$t = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} \tag{3.10}$$

Compare this to a critical t-value. If the p-value is small (e.g., ¡ 0.05), we reject H_0 — indicating the predictor is significant.

5. Hypothesis Testing for the Overall Model:

Sometimes, we are not just interested in individual predictors, but want to test whether the model as a whole is useful.

We test:

$$\boldsymbol{H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0} \quad \text{(none of the predictors help)} \boldsymbol{H_1 :} \text{ At least one } \boldsymbol{\beta_j \neq 0}$$

In other words, we are checking if the model explains significantly more variation than just using the mean of $\boldsymbol{Y}$ (i.e., no predictors).

This is done using the **F-statistic**:

$$\boldsymbol{F = \frac{(\text{TSS} - \text{RSS})/p}{\text{RSS}/(n-p-1)} = \frac{\text{Explained variance per predictor}}{\text{Unexplained variance per observation}}} \tag{3.11}$$

Where: - TSS: Total Sum of Squares = $\sum (y_i - \bar{y})^2$ - RSS: Residual Sum of Squares - $\boldsymbol{p}$: number of predictors - $\boldsymbol{n}$: number of observations

The F-statistic compares the model with predictors vs a model with no predictors. A high F-value means that adding predictors significantly improves the model. We then check the p-value associated with F.

If p-value is small (e.g., ¡ 0.05), we reject $\boldsymbol{H_0}$ — meaning that at least one predictor is useful.

This test is especially important in **multiple linear regression**, where we have many predictors. Even if none of them are individually significant, they may still have a combined effect.

6. Summary: Simple vs Multiple Regression

- In **simple linear regression**, we check if the single predictor is significant using the $\boldsymbol{t}$-test. - In **multiple linear regression**, we: - Test each coefficient using its own $\boldsymbol{t}$-test - Also test the model as a whole using the $\boldsymbol{F}$-statistic

All these metrics help us decide: - Which variables are statistically meaningful - Whether the model explains enough variation - And how much uncertainty there is in our estimates

This assessment is a crucial part of building robust regression models and interpreting their output correctly.

3.1.3 Assessing the Model Accuracy

Once we fit a linear regression model by estimating the coefficients $\hat{\boldsymbol{\beta}}$, the next important task is to evaluate how well the model is performing. In other words, we want to measure how close the predicted values $\hat{\boldsymbol{y}}$ are to the actual values $\boldsymbol{y}$.

This is called **assessing the model's accuracy**, and we do it using three common metrics:

- **RSS (Residual Sum of Squares)**
- **MSE (Mean Squared Error)**

- **R^2 Score (Coefficient of Determination)**

These metrics help us quantify how much of the variability in the data our model has captured.

1. Residual Sum of Squares (RSS)

The **RSS** is the total squared difference between the actual values y_i and the predicted values $\hat{y}_i$:

$$\text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 \tag{3.12}$$

- RSS measures the total amount of error left unexplained by the model. - Lower RSS means the model fits the data better. - RSS is always positive, and close to zero means predictions are very accurate.

2. Mean Squared Error (MSE)

To make RSS independent of sample size, we divide it by the number of observations:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = \frac{\text{RSS}}{n} \tag{3.13}$$

In practice, we often compute the **Training MSE** and the **Test MSE**: - **Training MSE**: How well the model fits the training data. - **Test MSE**: How well the model performs on unseen data (more important for real-world prediction).

A lower MSE means better accuracy, but too low MSE on training data may also suggest **overfitting**.

3. R^2 Score (Coefficient of Determination)

The **R^2 score** tells us how much of the total variability in the output y is explained by our model. It is defined as:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}} \tag{3.14}$$

Where:

$$\text{TSS} = \sum_{i=1}^{n} (y_i - \bar{y})^2 \quad \text{(Total Sum of Squares)}$$

Interpretation: - $R^2 = 1$: Perfect fit — the model explains all variability. - $R^2 = 0$: Model explains nothing — as bad as using the mean $\bar{y}$. - $R^2 > 0.7$: Generally considered good (but depends on domain).

R^2 is unitless and easy to interpret: - Closer to 1 $\rightarrow$ better performance. - Negative values $\rightarrow$ model is worse than just using the mean of y.

How to Interpret These Metrics Together:

- A good model should have: - **Low RSS** - **Low MSE** on test data - **High R^2** value (close to 1) - But we should also check for **overfitting**. A very low training MSE with high test MSE means the model learned the noise instead of the pattern.

Application to Simple vs Multiple Linear Regression:

- In **simple linear regression**, the interpretation is straightforward since we only have one predictor. - In **multiple linear regression**, these metrics help evaluate how all predictors together influence the response.

Additionally, in multiple regression: - R^2 always increases when we add more predictors. - So we also use **Adjusted R^2** (introduced later), which accounts for number of predictors.

These metrics give us a quantitative way to say: "How well is our model working?" and allow us to compare between models and make better decisions.

3.2 Linear Regression with Qualitative Predictors

So far, we have assumed that all the predictors in our regression models are quantitative. However, in real-world problems, it is very common to encounter **qualitative (categorical) variables** — such as gender, region, marital status, or whether someone is a student or homeowner.

To apply linear regression when some predictors are qualitative, we cannot directly use text labels or categories in the model. Instead, we must convert these categories into numerical values using a process called **dummy variable encoding** (also known as **one-hot encoding**).

1. Predictors with Two Levels

If a qualitative predictor has only two categories (e.g., "Yes" or "No"), we can simply use a binary dummy variable:

$$x_i = \begin{cases} 1 & \text{if the } i\text{-th person belongs to the category (e.g., owns a house)} \\ 0 & \text{if not (e.g., does not own a house)} \end{cases}$$

The linear regression model becomes:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

- β_0 represents the average response for the baseline group (coded as 0). - β_1 represents the difference in average response between the two categories.

This way, linear regression can easily capture the effect of a binary categorical variable.

2. Predictors with More Than Two Levels

If a qualitative variable has more than two levels (e.g., region = East, West, South), we need to create multiple dummy variables — one fewer than the total number of categories.

For example:

$$\text{region[South]} = \begin{cases} \mathbf{1} & \text{if from South} \\ \mathbf{0} & \text{otherwise} \end{cases} \qquad \text{region[West]} = \begin{cases} \mathbf{1} & \text{if from West} \\ \mathbf{0} & \text{otherwise} \end{cases}$$

No dummy variable is created for "East" — this becomes the **baseline category**. The regression model is:

$$y_i = \beta_0 + \beta_1 \cdot \text{South}_i + \beta_2 \cdot \text{West}_i + \varepsilon_i$$

Here: - β_0 = mean response for the baseline group (East) - β_1, β_2 = how much the response differs from East for South and West, respectively.

3. Interpretation and Modeling

- The choice of baseline category is arbitrary and does not change predictions — only the interpretation of coefficients. - For each qualitative variable, we always use $k - 1$ dummy variables where k is the number of levels. - We can include both quantitative and qualitative variables together in the same multiple linear regression model.

Conclusion:

Qualitative variables play an important role in many regression problems. Using dummy variables, we can apply linear regression to datasets that include such predictors without any difficulty. The key is to carefully choose the coding and correctly interpret the coefficients based on the baseline category.

This technique allowed me to include features like "student status" or "region" while modeling credit card balance or other responses during my internship projects.

3.3 Advantages & Disadvantages

Linear regression is one of the most widely used techniques in statistical modeling and supervised learning — and for good reason. After exploring all aspects of linear regression, including estimation, evaluation, and handling both quantitative and qualitative predictors, it's useful to reflect on the overall strengths and limitations of this method.

Advantages of Linear Regression

- **Simplicity and Interpretability:** Linear regression has a straightforward mathematical structure that makes it easy to implement and interpret. Each coefficient directly reflects the effect of a predictor on the response, which helps in understanding relationships between variables.
- **Fast and Computationally Efficient:** Since the solution has a closed-form (via the Normal Equation), linear regression is very efficient in terms of computation, especially when the number of predictors is small or moderate.
- **Works Well for Linearly Separable Data:** When the relationship between predictors and response is approximately linear, linear regression performs very well and gives accurate and reliable predictions.

- **Foundation for Many Advanced Methods:** Linear regression forms the base for many machine learning and statistical techniques, such as ridge regression, lasso, generalized linear models, and even logistic regression. Understanding it deeply helps in learning more complex models.
- **Probabilistic Interpretation:** With Gaussian assumptions, we can also perform statistical inference such as confidence intervals, hypothesis testing, and error analysis, which adds reliability to the model's results.

Disadvantages of Linear Regression

- **Assumes a Linear Relationship:** One of the biggest limitations is that linear regression assumes a linear relationship between inputs and output. If the true pattern in the data is non-linear, the model will underfit and fail to capture important structures.
- **Sensitive to Outliers:** Linear regression is highly sensitive to outliers, which can strongly influence the model fit and distort predictions or interpretations.
- **Multicollinearity Issues:** When predictors are highly correlated with each other, it becomes difficult to estimate their individual effects accurately. This can lead to unstable coefficient estimates and inflated standard errors.
- **Fails in High-Dimensional Settings:** When the number of predictors is large compared to the number of observations, linear regression performs poorly. It can overfit, generalize badly, and become unstable.
- **Limited Flexibility:** Linear regression cannot automatically capture complex patterns or interactions in the data. It requires manual feature engineering or extensions like polynomial terms, which may still be insufficient.

Why We Need Model Selection, Regularization, and Non-Linear Methods

Due to the limitations of standard linear regression, especially when working with real-world datasets that may contain many predictors, multicollinearity, or non-linear patterns, it becomes essential to move beyond basic linear models.

- **Model Selection Techniques** help us choose the most relevant subset of predictors (e.g., best subset selection, forward/backward stepwise), improving both interpretability and performance.
- **Regularization Methods** like **Ridge** and **Lasso** add penalty terms to control overfitting and stabilize the model when predictors are highly correlated or when $\boldsymbol{p > n}$ (more features than observations).
- **Non-Linear Models** such as **regression trees, splines, or ensemble methods** allow us to capture more complex relationships and interaction effects that a linear model cannot express.

In the upcoming chapters, I explored these advanced techniques in depth to overcome the challenges observed in linear regression. These extensions allowed me to build models that are more accurate, flexible, and better suited for complex real-world datasets.

4. Classification

In the previous chapter, I studied **linear regression**, which is used to predict a **quantitative output** based on one or more input features. However, in many real-world applications, the output variable is not numeric but **qualitative or categorical**. For example, predicting whether a transaction is **fraudulent or not**, whether a patient has a **disease or not**, or identifying the type of object (e.g., **cat, dog, car**) in an image — all are classification problems.

In classification, the goal is to predict the **class label** or **category** of an observation based on the input features. For example, given a person's income and credit card balance, can we predict whether they will **default** on their payment (Yes/No)? This is a binary classification problem.

Classification models output either:

- A direct predicted class label (e.g., 0 or 1), or
- A **probability score** for each class, from which we assign the most likely class.

Why Not Use Linear Regression for Classification?

It may seem like we could simply apply linear regression and treat classes as numerical labels (e.g., 0 = No, 1 = Yes). But this approach has serious problems:

- **No valid probability range:** Linear regression can give predicted values less than 0 or greater than 1, which are invalid if we want to interpret them as probabilities.
- **No control for class boundaries:** Linear regression does not handle classification boundaries properly. It assumes a linear relationship between input and output, which may not separate the classes well.
- **Multiclass problems break the model:** If we have more than two classes (e.g., stroke, overdose, seizure), using numbers like 1, 2, 3 introduces artificial ordering. This can mislead the model because these numbers imply a ranking which doesn't exist.
- **Misleading interpretation:** Coefficients from a regression model on a categorical target do not reflect odds or probabilities accurately — making interpretation unreliable.

Even though linear regression might give rough estimates or rankings in a binary case, it is not suitable for producing reliable **classification decisions or probability estimates**. Instead, we need classification-specific models that are designed to handle **categorical outputs**, such as **logistic regression**, which we discuss next.

4.1 Logistic Regression

In regression problems, we model a continuous output variable $\boldsymbol{Y}$ based on a set of inputs $\boldsymbol{X_1, X_2, ..., X_p}$. However, in many real-world situations, the output variable is **qualitative or categorical**. For example, whether a customer defaults on a credit card payment (Yes or No), or whether a tumor is malignant or benign. These are **binary classification problems**.

In such cases, traditional linear regression is not a suitable choice because:

- It assumes a continuous response and can predict values outside the $\boldsymbol{[0, 1]}$ probability range.

- It fails to model the true relationship between input features and classification probabilities.

To solve this, we use **Logistic Regression** — a classification algorithm used to predict the probability that a given input belongs to a particular class.

The Goal:

We aim to model the probability:

$$p(X) = \Pr(Y = 1 \mid X)$$

where Y is the binary response variable ($\mathbf{0}$ or $\mathbf{1}$) and X is the input vector.

Instead of fitting $p(X)$ directly, logistic regression models the **log-odds** (also known as the logit):

$$\log\left(\frac{p(X)}{1 - p(X)}\right) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p$$

This equation is called the **logistic function in logit form**. It transforms the linear combination of input features into a quantity that can be interpreted as a probability between 0 and 1.

Probability Representation:

By solving the above logit equation for $p(X)$, we get the logistic (sigmoid) function:

$$p(X) = \frac{e^{\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p}}{1 + e^{\beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p}}$$

This function always outputs values between 0 and 1, which makes it ideal for modeling classification probabilities.

Decision Rule:

Once we have predicted the probability $p(X)$ for a new observation, we classify it into one of the two classes using a threshold (typically 0.5):

$$\hat{Y} = \begin{cases} \mathbf{1} & \text{if } p(X) > \mathbf{0.5} \\ \mathbf{0} & \text{otherwise} \end{cases}$$

Interpretation of Coefficients:

Each coefficient β_j in logistic regression represents the change in the **log-odds** of the outcome for a one-unit increase in X_j, keeping other predictors constant. If β_j is positive, the odds of $Y = 1$ increase as X_j increases.

- The odds of an event are defined as odds $= \frac{p(X)}{1-p(X)}$
- The log of odds is the logit, and logistic regression assumes a linear relationship between the log-odds and the inputs

Preprocessing Requirement – Standardization:

In my implementation, I found it essential to **standardize the input features** before training the logistic regression model. This is especially important when input variables are on different scales. Standardization ensures that each feature contributes equally to the optimization process and avoids numerical instability.

Before prediction, I made sure to **standardize new data using the same mean and standard deviation** computed from the training set. This maintains consistency between training and prediction. However, rescaling the coefficients is not necessary for prediction — only if we want to interpret them in the original units.

When to Use Logistic Regression:

- When the response variable is binary (Yes/No, True/False, 0/1)
- When you want to estimate probabilities directly
- When model interpretability is important

Limitations:

- It only models linear decision boundaries (in log-odds space)
- It cannot be used directly for multi-class problems (extensions like multinomial logistic regression are used)
- Outliers can sometimes influence the estimation

In this internship, I used logistic regression to model binary outcomes across multiple datasets, and I implemented the algorithm manually in Python using **Maximum Likelihood Estimation (MLE)**, rather than relying on libraries. This helped me understand the mathematical intuition and the optimization process deeply. In the next section, I explain how the model coefficients are estimated.

4.1.1 Estimating the Coefficients

In linear regression, we estimate coefficients using the **least squares method**. However, this approach does not work in logistic regression because the response variable is binary (0 or 1), and we are modeling probabilities — not a continuous output.

Instead, **logistic regression uses Maximum Likelihood Estimation (MLE)** to estimate the model coefficients $\boldsymbol{\beta_0, \beta_1, \ldots, \beta_p}$.

Likelihood Function:

Let:

- $\boldsymbol{y_i \in \{0, 1\}}$ be the observed outcome for the $\boldsymbol{i^{th}}$ observation
- $\boldsymbol{x_i = [x_{i1}, x_{i2}, \ldots, x_{ip}]}$ be the input vector for that observation
- $\boldsymbol{p(x_i)}$ be the predicted probability that $\boldsymbol{y_i = 1}$

Then the likelihood of observing the data given the model is:

$$L(\beta) = \prod_{i=1}^{n} p(x_i)^{y_i} (1 - p(x_i))^{1-y_i}$$

This equation means:

- If $y_i = 1$, the term becomes $p(x_i)$
- If $y_i = 0$, the term becomes $1 - p(x_i)$

Log-Likelihood Function:

Instead of maximizing the product (which is hard to compute), we take the logarithm of the likelihood — this turns the product into a sum:

$$\ell(\beta) = \sum_{i=1}^{n} [y_i \log(p(x_i)) + (1 - y_i) \log(1 - p(x_i))]$$

This is called the **log-likelihood**, and we now want to find the values of β that **maximize this function**.

Optimization:

The log-likelihood function is maximized using iterative numerical algorithms such as:

- **Gradient Descent**
- **Newton-Raphson Method**
- **Iteratively Reweighted Least Squares (IRLS)** — commonly used in logistic regression

These methods update the coefficients step-by-step until they converge to values that best fit the data.

Output of the Estimation:

- Each coefficient $\hat{\beta}_j$ represents the change in the **log-odds** of the outcome for a one-unit increase in X_j, holding all other variables constant.
- The sign of $\hat{\beta}_j$ tells us the direction of influence:
  - Positive ⇒ increases the likelihood of class 1
  - Negative ⇒ decreases the likelihood of class 1

Example Interpretation:

If $\hat{\beta}_1 = 0.8$, then a one-unit increase in X_1 multiplies the odds of $Y = 1$ by $e^{0.8} \approx 2.22$. That means the event becomes more than twice as likely.

Use in Simple and Multiple Logistic Regression:

- In **simple logistic regression** (only one predictor), the model becomes:

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 x$$

  and the estimation gives us two coefficients β_0 and β_1.

- In **multiple logistic regression**, the model is:

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p$$

  and we estimate all $p+1$ coefficients together using MLE.

In summary, estimating coefficients in logistic regression is all about finding the parameters that maximize the likelihood of correctly predicting the outcomes. This is what allows logistic models to output reliable probability estimates and classification decisions.

4.1.2 Hypothesis Testing of Coefficients

In logistic regression, once we estimate the coefficients $\beta_0, \beta_1, \ldots, \beta_p$ using Maximum Likelihood Estimation (MLE), the next step is to assess the significance of each coefficient. This helps us understand whether each predictor has a meaningful contribution in determining the outcome variable.

Objective:

For each predictor X_j, we want to test whether the associated coefficient β_j is statistically significantly different from zero.

Null and Alternative Hypotheses:

For each coefficient β_j:

$$H_0 : \beta_j = 0 \quad \text{(no effect)} H_1 : \beta_j \neq 0 \quad \text{(significant effect)}$$

If $\beta_j = 0$, it means that the predictor X_j does not contribute to predicting the log-odds of the response.

Standard Error and Test Statistic:

After fitting the logistic regression model, we obtain:

- $\hat{\beta}_j$: estimated coefficient
- $SE(\hat{\beta}_j)$: standard error of $\hat{\beta}_j$

We compute the test statistic for each coefficient using:

$$z_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

Under the null hypothesis, this statistic approximately follows a standard normal distribution $N(0, 1)$ when the sample size is large.

P-value and Decision Rule:

Using the z-value, we compute the two-sided p-value:

$$p = 2 \cdot P(Z > |z_j|)$$

If the p-value is small (commonly less than 0.05), we reject the null hypothesis and conclude that β_j is significantly different from zero, meaning X_j is an important predictor.

95% Confidence Interval for Coefficients:

We can also construct a confidence interval for each β_j using the formula:

$$\hat{\beta}_j \pm z_{\alpha/2} \cdot SE(\hat{\beta}_j) \quad \text{(with } z_{0.975} = 1.96 \text{ for 95\% CI)}$$

This interval gives a range of values within which we are 95

Interpreting in Logistic Regression Context:

Keep in mind:

- Coefficients represent **log-odds**.
- Exponentiating the coefficients gives **odds ratios**, i.e., e^{β_j} shows how much the odds of $Y = 1$ change for a one-unit increase in X_j.
- A significant p-value indicates that the variable has a measurable impact on the probability of success (class 1).

Simple vs Multiple Logistic Regression:

- In **simple logistic regression** (one predictor), hypothesis testing checks whether that single variable is related to the response.
- In **multiple logistic regression**, we test each variable while adjusting for the presence of other predictors. This controls for confounding and gives a more reliable picture.

Overall Model Significance (Global Hypothesis Test):

Just as in linear regression, we may want to test whether **all predictors together have any effect**. This is done via the **likelihood ratio test (LRT)**.

We compare two models:

- **Null Model:** contains only intercept (β_0)
- **Full Model:** includes all predictors

The test statistic is:

$$G = -2 \cdot (\log L_{\text{null}} - \log L_{\text{full}})$$

This G-statistic follows a chi-squared distribution with degrees of freedom equal to the number of predictors:

$$G \sim \chi_p^2$$

If the p-value is small, we reject $H_0 : \beta_1 = \beta_2 = \cdots = \beta_p = 0$, which means that **at least one predictor is useful**.

Summary of Testing Process:

1. Estimate coefficients using MLE.
2. Calculate standard errors of the coefficients.
3. Compute z-statistics and p-values.
4. Check 95% confidence intervals.
5. Use likelihood ratio test for overall model significance.

In my internship, I followed this entire process in every logistic regression model. This helped me understand which variables significantly affect the classification and how confident we are about the model's decisions. I also verified all results by comparing manual calculations to built-in Python libraries (for validation only).

4.1.3 Assessing Model Accuracy in Logistic Regression

After fitting a logistic regression model and estimating its coefficients, the next essential step is to evaluate how well the model performs on classification tasks. In logistic regression, we do not predict a continuous output — instead, we predict probabilities and then use them to classify each observation into a binary class (e.g., 0 or 1, No or Yes).

To assess classification performance, several evaluation tools are used. Each one provides different insights into model quality.

1. Confusion Matrix

The confusion matrix is a table that summarizes the performance of a classification model by comparing the predicted labels with the actual labels.

$$\begin{bmatrix} \textbf{True Negative (TN)} & \textbf{False Positive (FP)} \\ \textbf{False Negative (FN)} & \textbf{True Positive (TP)} \end{bmatrix}$$

- **True Positive (TP):** Model predicts class 1 and it's actually 1.
- **True Negative (TN):** Model predicts class 0 and it's actually 0.
- **False Positive (FP):** Model predicts class 1 but it's actually 0.
- **False Negative (FN):** Model predicts class 0 but it's actually 1.

The confusion matrix helps us calculate key performance metrics.

2. Accuracy

Accuracy is the simplest metric and measures the overall percentage of correct predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

While accuracy is intuitive, it can be misleading if the dataset is imbalanced. For example, if 95

3. Precision, Recall, and F1 Score

To understand model behavior better, we use additional metrics:

- **Precision:**
$$\text{Precision} = \frac{TP}{TP + FP}$$
It measures how many predicted positives were actually correct.
- **Recall (Sensitivity or True Positive Rate):**
$$\text{Recall} = \frac{TP}{TP + FN}$$
It measures how many actual positives the model correctly identified.
- **F1 Score:**
$$\text{F1} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$
It is the harmonic mean of precision and recall, used when we need to balance both.

4. ROC Curve and AUC Score

The Receiver Operating Characteristic (ROC) curve is a graphical plot that shows the trade-off between the **True Positive Rate (Recall)** and the **False Positive Rate** across various threshold values.

- The **False Positive Rate (FPR)** is defined as:
$$\text{FPR} = \frac{FP}{FP + TN}$$
- The ROC curve helps visualize how well the model distinguishes between classes.
- A curve closer to the top-left indicates better performance.

AUC Score (Area Under the Curve):

$$\text{AUC} = \text{Area under the ROC curve} \quad (0 \leq \text{AUC} \leq 1)$$

- AUC of 0.5: model is random (no discrimination)
- AUC close to 1: excellent classification
- AUC below 0.5: worse than random (very rare)

In my internship, I used both ROC plots and AUC scores to compare logistic models, especially when dealing with imbalanced classes.

5. Classification Report

The classification report combines all the above metrics — accuracy, precision, recall, and F1 score — for each class.

- It is especially helpful when classes are imbalanced.
- It gives a clearer understanding of model performance for each class label.

Summary of Evaluation Metrics:

- Use **confusion matrix** to understand true vs false predictions.
- Use **accuracy** as a general metric.
- Use **precision, recall, and F1** for class-specific analysis.
- Use **ROC curve and AUC** to measure probability-based ranking and class separation.
- Use **classification report** for a detailed performance summary.

In my internship, I applied all these evaluation techniques to every classification model I implemented. This allowed me not just to build models, but to fully understand how well they were working and where they might fail — especially in the case of imbalanced data or threshold-based decisions.

4.1.4 Advantages and Disadvantages of Logistic Regression

After studying and implementing logistic regression during this internship, I realized that although the method looks simple at first, it actually has a lot of practical advantages that make it a powerful baseline model in classification problems. However, like any technique, it also has its limitations, and knowing these helps us decide when to move toward more advanced models.

Advantages:

- **Interpretability:** One of the most important advantages is that logistic regression gives interpretable coefficients. Each coefficient explains how the log-odds of the output changes with a unit change in the predictor.
- **Probabilistic Output:** It directly estimates the probability of a class label. This makes it very useful in cases where we not only want a prediction but also a confidence score.
- **Computational Efficiency:** It is easy to implement and computationally lightweight, which makes it suitable for large-scale problems and quick baselines.
- **Theoretical Foundation:** Logistic regression has a well-understood mathematical foundation. Its loss function is convex, which guarantees that the optimization process using gradient-based methods will converge to the global minimum.
- **Works Well with Linearly Separable Data:** When the decision boundary between classes is linear, logistic regression performs very well.

- **Extensible to Multiclass Problems:** Logistic regression can be extended to handle multi-class classification using softmax (multinomial logistic regression).

Disadvantages:

- **Linear Decision Boundary:** Logistic regression assumes that the log-odds are a linear function of input features. So, it fails to capture complex non-linear relationships between variables.
- **Sensitive to Multicollinearity:** If the input features are highly correlated, the estimates of the coefficients become unstable, and the model can behave poorly.
- **Outlier Sensitivity:** Logistic regression is quite sensitive to outliers which can disproportionately affect the model parameters.
- **Requires Feature Engineering:** Logistic regression heavily depends on proper feature selection and transformations to perform well, especially on real-world datasets.
- **Poor Performance on Imbalanced Datasets:** If the dataset is highly imbalanced (e.g., many more 0s than 1s), logistic regression may be biased toward the majority class and fail to detect the minority class properly.

How to Handle the Limitations:

- For capturing non-linear patterns, we can add polynomial features or move to models like **decision trees** or **SVMs**.
- To reduce multicollinearity, we can apply dimensionality reduction techniques like **PCA**, or use **regularization** (Lasso or Ridge).
- To reduce the impact of outliers, we can use robust methods or clean the data using outlier detection techniques.
- For imbalanced data, techniques like **resampling**, **SMOTE**, or using **class weights** in the loss function can help.

Overall, logistic regression still remains a very practical, interpretable, and useful method to start with in binary classification tasks. In this internship, it gave me a solid foundation for understanding classification from a probabilistic perspective and helped me connect theory with actual prediction tasks across datasets.

4.2 K-Nearest Neighbors (K-NN) Classification

The k-Nearest Neighbors (K-NN) algorithm is one of the most intuitive and non-parametric methods for classification tasks. Unlike logistic regression, K-NN does not assume any specific functional form for the relationship between predictors and response. Instead, it makes predictions based directly on the training data by observing the "neighborhood" of a test point.

K-NN is a lazy learning algorithm, meaning it does not explicitly build a model during training. Instead, all the computation happens at the time of prediction.

Working Principle of K-NN

Given a test observation, the K-NN algorithm performs the following steps:

- Compute the distance (usually Euclidean) between the test point and every point in the training dataset.
- Select the k closest training observations.
- Predict the class by taking a **majority vote** among the k nearest neighbors.

If the majority of the k neighbors belong to class 1, then the algorithm predicts class 1. If the majority belong to class 0, then class 0 is predicted.

Choice of k

The parameter k is a critical component of K-NN:

- **Small k (e.g., $k = 1$)**: Model becomes highly flexible, potentially overfitting the noise in training data.
- **Large k (e.g., $k = 20$)**: Model becomes more stable and less sensitive to noise but may underfit the data.

Choosing the right k involves cross-validation. I used techniques such as **k-fold cross-validation** to select the value of k that minimizes test classification error.

Distance Metric

The most common distance metric used in K-NN is Euclidean distance:

$$\text{Distance}(x, x_0) = \sqrt{(x_1 - x_{01})^2 + (x_2 - x_{02})^2 + \cdots + (x_p - x_{0p})^2}$$

Other distance metrics include Manhattan, Minkowski, or cosine distance — but Euclidean is generally used for low to moderate dimensions.

Decision Boundary and Flexibility

K-NN generates very flexible decision boundaries. The shape of the boundary depends on the value of k and the local distribution of training data.

- **Low k** $\rightarrow$ highly non-linear and wiggly boundary (high variance).
- **High k** $\rightarrow$ smoother, more stable boundary (high bias).

This bias–variance trade-off is central to the performance of K-NN, similar to what I studied in linear and logistic regression.

Probability Interpretation

K-NN can also give estimated probabilities instead of direct class labels. For a binary classification:

$$\hat{P}(Y = 1|x_0) = \frac{\text{Number of class 1 samples in } k \text{ neighbors}}{k}$$

This probability can be thresholded (e.g., at 0.5) to get a class prediction, similar to logistic regression.

Feature Scaling is Important

Since K-NN relies on distance, it is highly sensitive to the scale of features. For example, if one feature is measured in thousands and another in decimals, the larger scale feature will dominate the distance. Therefore:

- **Feature normalization or standardization** (e.g., z-score scaling) is necessary.

During my implementation, I always normalized features before applying K-NN using:

$$z = \frac{x - \mu}{\sigma}$$

Model Evaluation for K-NN

Since K-NN gives class predictions, I used the same set of classification metrics that I applied to logistic regression:

- **Confusion Matrix**
- **Accuracy, Precision, Recall, F1 Score**
- **ROC Curve and AUC Score**

I found that K-NN performs well when:

- The data is low-dimensional,
- There is enough data for each class,
- Proper scaling and k tuning is performed.

Strengths of K-NN

- Simple and intuitive to understand and implement.
- Naturally handles multi-class classification.
- No training phase — computationally cheap during training.
- Adaptable to complex decision boundaries.

Limitations of K-NN

- **Computationally expensive at prediction time**, especially for large datasets.
- Very sensitive to irrelevant or redundant features.
- Suffers from the "curse of dimensionality" — as dimensions increase, distance-based methods lose effectiveness.
- Requires proper feature scaling to perform well.

Conclusion

K-NN classification is a powerful non-parametric method, especially when the structure of data is complex or unknown. During my internship, I implemented K-NN from scratch using Python and applied it to various datasets including **Credit Card Default** and **Heart Disease**. I experimented with different k values, normalization techniques, and evaluated models using confusion matrix and AUC. While K-NN is not always the best choice for large or high-dimensional datasets, it gave me clear intuition about how local decision-making can work in classification problems.

5. Linear Model Selection and Regularization

In previous chapters, especially while working with **linear regression**, I used all available features to fit the model. But during my hands-on work, I started noticing that using all predictors doesn't always give the best results. The model fits the training data very well, but often fails on new, unseen data. This is a classic case of **overfitting**, where the model becomes too flexible and starts capturing noise instead of the actual signal.

One major reason for this is that some predictors may have little to no relationship with the response variable. Including such irrelevant features increases the model's **variance**, making it less stable and more prone to errors during generalization. At the same time, if we remove too many features or use only a small subset blindly, we risk **underfitting** — where the model is too simple to learn the true underlying patterns.

So, to build models that generalize well and are also interpretable, we need smart techniques to:

- Select the most informative predictors automatically,
- Reduce complexity without losing key information,
- Improve performance by balancing **bias-variance trade-off**,
- And avoid relying on manual feature selection.

That's where **model selection** and **regularization** methods become extremely important. These techniques allow us to either choose a good subset of features or apply penalties to reduce the influence of less important predictors. They help us control overfitting and build models that work well on both training and testing datasets.

In this chapter, I studied the following techniques :

- **Best Subset Selection**
- **Forward and Backward Stepwise Selection**
- **Ridge Regression (L2 Regularization)**
- **Lasso Regression (L1 Regularization)**
- **Elastic Net (a hybrid of L1 and L2)**
- **Dimension Reduction**

These techniques gave me deep insights into how to manage feature complexity and how different methods behave in different scenarios.

The core goal of this chapter is to build simpler, more accurate, and more interpretable models by intelligently selecting or regularizing the set of input predictors — instead of blindly using all variables available in the data.

5.1 Subset Selection

Subset selection is a method used to identify a smaller group of predictors from the full set that contribute the most to predicting the output. Instead of using all variables, we try to select the best combination of features that improves model performance and reduces complexity.

5.1.1 Best Subset Selection

Best Subset Selection is a method that helps us choose the best subset of predictors from a full set of input variables for building a linear regression model. Instead of using all the predictors, we try different combinations and pick the one that gives the best model performance based on a certain criterion (like RSS, Adjusted R^2, AIC, BIC, or test error).

How it works:

- For a dataset with p predictors, the algorithm fits all possible models with k predictors for $k = 1$ to p.
- For each value of k, it chooses the best model (with lowest RSS or highest Adjusted R^2).
- Finally, it compares the best models of all sizes and selects the one with the best overall performance on validation data.

Algorithm Steps:

1. Start with no predictor.
2. For $k = 1$ to p:
    - Fit all $\binom{p}{k}$ models with k predictors.
    - Select the one with the best performance (e.g., lowest RSS).
3. From the p models, choose the overall best one using cross-validation or another performance metric.

Advantages:

- Examines all possible models — gives the best subset for each model size.
- Easy to interpret and understand.
- Good starting point for small datasets with few predictors.

Disadvantages:

- Computationally expensive — becomes infeasible when p is large.
- Risk of overfitting, especially when we select models based on training error only.
- Cannot scale well with high-dimensional datasets.

It helped me understand the need for more scalable techniques like forward and backward stepwise selection, which I explore in the next sections.

5.1.2 Stepwise Selection

When the number of predictors becomes large, **Best Subset Selection** becomes computationally very expensive since it fits all possible combinations. To overcome this, we use **Stepwise Selection**, a more efficient approach that builds the model step-by-step by either adding or removing predictors based on a performance criterion.

1. Forward Stepwise Selection

Forward Selection starts with an empty model (no predictors) and adds variables one by one. At each step, the variable that improves the model the most (e.g., reduces RSS or improves R^2) is added.

Algorithm:

1. Start with the null model (only intercept, no predictors).
2. For each variable not in the model:
    - Fit a model by adding that variable.
    - Calculate performance (e.g., RSS, AIC, BIC).
3. Add the variable that improves the model the most.
4. Repeat steps 2–3 until:
    - No variable improves the model further, or
    - A maximum number of predictors is reached.

Advantages:

- Much faster than Best Subset Selection.
- Works well when number of predictors is large.
- Easy to implement and interpret.

Disadvantages:

- May miss the best overall model since it builds sequentially.
- Once a variable is added, it can't be removed — this can lead to suboptimal models.

2. Backward Stepwise Selection

Backward Selection starts with the full model (all predictors included) and removes the least useful variable at each step.

Algorithm:

1. Start with the full model (all p predictors).
2. For each variable in the model:
    - Fit a model by removing that variable.
    - Calculate performance (e.g., RSS, AIC, BIC).
3. Remove the variable that has the least impact (smallest degradation).
4. Repeat steps 2–3 until:
    - Removing any variable worsens the model significantly.

Advantages:

- Also faster than Best Subset Selection.
- Good when we suspect many variables are not useful.

Disadvantages:

- Requires the number of observations n to be much greater than number of predictors p (so that full model can be fit initially).
- Risk of removing variables that could be useful in combination with others.

3. Hybrid Stepwise Selection

Hybrid Selection (also known as bidirectional) combines both forward and backward steps. At each stage, we either add a new variable or remove an existing one, depending on which operation improves the model most.

Algorithm:

1. Start with an empty model or a simple model.
2. In each iteration:
    - Add the best variable not already in the model.
    - Remove any variable if it has become unhelpful.
3. Repeat this process of addition/removal until:
    - No addition or removal improves the model.

Advantages:

- Flexible and adapts better than forward or backward alone.
- Can produce better models in practice.

Disadvantages:

- Still heuristic and may not guarantee finding the global best model.
- Slightly more complex to implement.

What I Learned:

Through these techniques, I realized that stepwise methods strike a balance between model performance and computational feasibility.

Why Move Toward Regularization?

Even though subset selection methods are effective for small to medium feature sets, they may still lead to unstable models, especially when predictors are highly correlated. In such cases:

- Stepwise methods may include or exclude variables based on small data fluctuations.
- There's no penalty to prevent overfitting — they simply pick or drop variables.

To address this, we move to **Shrinkage Methods** like Ridge and Lasso, which not only help in feature selection (in case of Lasso) but also penalize large coefficients, leading to more stable and generalized models.

In the next section, I will explain how shrinkage methods work and how I implemented them during my internship.

5.2 Shrinkage Methods

While subset selection methods choose a subset of predictors, **shrinkage methods** take a different approach — they include all predictors but **shrink the magnitude of their coefficients**. This helps in reducing variance, preventing overfitting, and improving model stability, especially when we have multicollinearity or a large number of predictors.

Instead of completely removing predictors, these methods apply a penalty to the regression coefficients, forcing them to be smaller and often close to zero.

The main shrinkage techniques I studied and implemented are: **Ridge Regression**, **Lasso Regression** and **Elastic Net (a combination of Ridge and Lasso)**

In the following sections, I explain each method in detail, how they work mathematically.

5.2.1 Ridge Regression

In linear regression, when we have a large number of predictors — especially when some of them are highly correlated — the model can become unstable and overfit the training data. **Ridge Regression** is introduced to address this problem by adding a penalty that shrinks the coefficients and reduces model variance.

What is Ridge Regression?

Ridge regression is a **shrinkage method** that modifies the ordinary least squares (OLS) objective function by adding an L2 penalty term. This penalty discourages large coefficients, making the model more robust.

$$\text{Loss}_{\text{ridge}} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

Here, $\lambda \geq 0$ is a hyperparameter that controls the strength of the regularization. A larger λ leads to greater shrinkage of coefficients.

Why L2 Penalty? What's the Geometry?

The L2 penalty shrinks the coefficients towards zero but never exactly to zero. It keeps all variables in the model but reduces their impact proportionally. Geometrically, the constraint region is a circle (or hypersphere), and the ridge solution lies where this region touches the ellipsoidal contours of the least squares error.

Why Standardization is Required?

Before applying ridge regression, we **standardize all predictors** (except the intercept) so that each variable has mean zero and unit variance. This ensures that the penalty is applied fairly across all variables, regardless of scale.

After fitting the model on standardized predictors, we calculate the intercept using:

$$\beta_0 = \bar{y} - \sum_{j=1}^{p} \beta_j \bar{x}_j$$

Mathematical Derivation: Estimating Ridge Coefficients

Let X be the $n \times p$ matrix of standardized predictors, and y be the response vector. Ridge regression minimizes the following objective:

$$\text{RSS}_{\text{ridge}} = (y - X\beta)^T (y - X\beta) + \lambda \beta^T \beta$$

Taking derivative with respect to β:

$$-2X^T(y - X\beta) + 2\lambda\beta = 0 \Rightarrow X^T X\beta + \lambda I\beta = X^T y \Rightarrow \beta_{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y$$

This formula gives us the ridge estimates. The term λI ensures that the matrix is invertible, even if $X^T X$ is not, thus handling multicollinearity effectively.

How to Make Predictions? Do We Need to Rescale Coefficients?

After estimating β_{ridge} using standardized predictors, predictions on new (unseen) data should be made carefully:

- Standardize the new input features using the **same mean and standard deviation** used during training.
- Use the ridge coefficients on the standardized features and add the intercept calculated above.

There is **no need to rescale the coefficients manually**, because we apply the standardization step to the input data before prediction. However, if you want to interpret the coefficients in the original scale, then you may transform them back by reversing the standardization — but for prediction, you simply standardize the input again.

How to Choose the Best λ?

The tuning parameter λ is selected via **cross-validation**. We try different values of λ and pick the one that minimizes the cross-validated prediction error, usually using k-fold CV.

Advantages of Ridge Regression:

- Reduces overfitting and handles multicollinearity effectively.
- Keeps all predictors in the model — useful when many features contribute a little.

- Has a closed-form solution and is computationally efficient.
- Can significantly reduce test error compared to OLS when p is large or predictors are correlated.

Disadvantages of Ridge Regression:

- Does not perform variable selection — all coefficients are shrunk, but none become exactly zero.
- Interpretation becomes hard when many small, non-zero coefficients are present.

If we want a sparse model that automatically selects a subset of predictors (by forcing some coefficients to be exactly zero), we move to another method — **Lasso Regression**, which is discussed next.

5.2.2 Lasso Regression

Why Lasso Regression?

Ridge regression helps control overfitting by shrinking coefficients using L2 penalty. But one limitation of Ridge is that it does not perform **feature selection** — it keeps all predictors in the model. In many situations, we prefer a simpler model where only a few variables are actually used. To solve this, we use **Lasso Regression**.

What is Lasso Regression?

Lasso (Least Absolute Shrinkage and Selection Operator) regression is a regularization method that uses an **L1 penalty** instead of L2. The objective function becomes:

$$\text{Loss}_{\text{lasso}} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{p} |\beta_j|$$

Here, $\lambda \geq 0$ is the regularization parameter. The key difference is the use of the absolute value (L1 norm), which leads to some coefficients becoming **exactly zero**, hence performing variable selection.

L1 Norm and Geometry Intuition:

The L1 penalty $\sum |\beta_j|$ creates a diamond-shaped constraint region (in 2D). When we solve for the coefficients, the ellipsoids from the loss function often hit the edges or corners of this diamond, causing some coefficients to become zero. This is what allows Lasso to eliminate unimportant features.

Standardization is Required Before Fitting:

Just like Ridge, in Lasso we must **standardize the predictors** to have mean zero and unit variance before fitting. This ensures that the penalty treats all predictors equally.

How to Estimate Lasso Coefficients:

Unlike Ridge, Lasso does not have a simple closed-form solution due to the absolute value. So we use optimization techniques like **Coordinate Descent** to minimize the objective.

Let $\boldsymbol{X}$ be the matrix of standardized predictors and $\boldsymbol{y}$ the centered response. The lasso problem becomes:

$$\hat{\beta}^{lasso} = \arg\min_{\beta} \left[(y - X\beta)^T (y - X\beta) + \lambda \sum_{j=1}^{p} |\beta_j| \right]$$

Coordinate Descent updates one coefficient at a time while keeping others fixed. Each $\boldsymbol{\beta_j}$ is updated using a **soft-thresholding operator**:

$$\hat{\beta}_j \leftarrow \text{sign}(z_j) \cdot \max(|z_j| - \lambda, 0)$$

where z_j is the partial residual correlation with feature $\boldsymbol{j}$.

This iterative process continues until convergence. The result is that some $\boldsymbol{\beta_j}$ will become exactly zero depending on $\boldsymbol{\lambda}$.

How to Calculate Intercept:

After fitting the Lasso model with standardized predictors, we calculate the intercept separately (in original scale) using:

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^{p} \hat{\beta}_j \cdot \bar{x}_j$$

Here, $\bar{x}_j$ and $\bar{y}$ are the means of the original predictors and response before standardization.

How to Tune the Hyperparameter λ (or α):

The choice of $\boldsymbol{\lambda}$ is crucial. If it's too large, all coefficients may shrink to zero; if it's too small, the model may overfit. To choose the optimal $\boldsymbol{\lambda}$, we typically use:

- **Cross-validation (CV):** Most commonly, $\boldsymbol{k}$-fold CV is used. We fit Lasso models for a grid of $\boldsymbol{\lambda}$ values and select the one with the lowest validation error.
- **Grid Search:** A range of $\boldsymbol{\lambda}$ values is tested (on a log scale), and the best performing one is selected.

Rescaling Coefficients — Is It Needed?

No, you don't need to rescale coefficients manually if you apply the same standardization to new data as during training. Just remember:

- Standardize input data before prediction using training means and std.
- Use learned $\hat{\boldsymbol{\beta}}$ and $\hat{\boldsymbol{\beta}}_{\boldsymbol{0}}$ to predict.

However, for interpretation in the original scale (not prediction), you can reverse the standardization.

Prediction on New Data:

To make predictions on new samples:

- Standardize new features using the same training statistics.
- Use the model to compute:

$$\hat{y}_{\text{new}} = \hat{\beta}_0 + \sum_{j=1}^{p} x^*_{j,\text{new}} \cdot \hat{\beta}_j$$

Advantages of Lasso:

- Performs automatic **feature selection** by setting coefficients to zero.
- Produces sparse and interpretable models.
- Helps reduce overfitting in high-dimensional datasets.

Disadvantages of Lasso:

- When predictors are highly correlated, Lasso may arbitrarily pick one and ignore others.
- It can shrink large coefficients too much.
- For $p > n$, it can select at most n predictors.

Because Lasso may behave poorly when variables are strongly correlated, we next explore a method that combines the strengths of both Ridge and Lasso — the **Elastic Net**, discussed in the next section.

5.2.3 Elastic Net

Why Elastic Net?

Lasso regression is powerful for feature selection, but it has some limitations — especially when there are **highly correlated predictors**. In such cases, Lasso tends to arbitrarily select one variable and ignore others, even if all are important. On the other hand, Ridge regression keeps all variables but cannot perform feature selection. To overcome these limitations, we use a method that combines the strengths of both Ridge and Lasso — called the **Elastic Net**.

What is Elastic Net?

Elastic Net combines the penalties of Lasso (L1) and Ridge (L2) into a single model. The objective function becomes:

$$\text{Loss}_{\text{EN}} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 + \lambda \left[\alpha \sum_{j=1}^{p} |\beta_j| + (1 - \alpha) \sum_{j=1}^{p} \beta_j^2 \right]$$

Here:

- λ controls the overall strength of regularization,
- $\alpha \in [0, 1]$ controls the mix between Lasso and Ridge:
  - $\alpha = 1 \Rightarrow$ Lasso,
  - $\alpha = 0 \Rightarrow$ Ridge,
  - $0 < \alpha < 1 \Rightarrow$ Elastic Net.

Geometric Interpretation:

Elastic Net's constraint region is a compromise between the diamond (L1) and the circle (L2). As a result, it has the ability to:

- Perform feature selection like Lasso,
- Retain correlated variables like Ridge.

Standardization Before Fitting:

Just like in Ridge and Lasso, we must standardize the input predictors (zero mean, unit variance) before fitting the Elastic Net model. This ensures the penalty treats all features equally.

How to Estimate Coefficients in Elastic Net:

Elastic Net also doesn't have a closed-form solution due to the L1 part. So we use optimization techniques like **coordinate descent** or **gradient-based solvers**. The minimization is performed iteratively while balancing both L1 and L2 penalties.

Intercept Calculation:

After fitting the model on standardized predictors, the intercept is computed as:

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^{p} \hat{\beta}_j \cdot \bar{x}_j$$

Where $\bar{x}_j$ and $\bar{y}$ are the original (non-standardized) feature and response means.

How to Make Predictions:

To predict for new data:

- First standardize the features using training mean and std.
- Then apply:

$$\hat{y}_{\text{new}} = \hat{\beta}_0 + \sum_{j=1}^{p} x^*_{j,\text{new}} \cdot \hat{\beta}_j$$

Do We Need to Rescale Coefficients?

No rescaling is needed for prediction — as long as standardization is done consistently on new data. However, if we want to interpret coefficients in the original scale, we can reverse the standardization manually.

How to Tune Hyperparameters λ and α:

Elastic Net involves two hyperparameters:

- $\boldsymbol{\lambda}$ — controls the regularization strength.
- $\boldsymbol{\alpha}$ — balances between Lasso and Ridge penalties.

To tune these:

- Use **grid search with cross-validation** over a range of $(\boldsymbol{\lambda}, \boldsymbol{\alpha})$ values.
- Select the pair that minimizes validation error (like Mean Squared Error or Cross-Validation loss).
- In Python, `ElasticNetCV()` from `sklearn` automates this process.

Advantages of Elastic Net:

- Works well with correlated features.
- Performs variable selection and coefficient shrinkage together.
- Flexible — acts like Lasso or Ridge depending on $\boldsymbol{\alpha}$.
- Can select more than $\boldsymbol{n}$ features when $\boldsymbol{p > n}$ (unlike Lasso).

Disadvantages of Elastic Net:

- Requires tuning of two hyperparameters — more complex than Ridge or Lasso alone.
- May be harder to interpret when $\boldsymbol{\alpha}$ is not clearly defined.

5.3 Dimension Reduction Methods

Why Dimension Reduction?

In high-dimensional datasets, where the number of predictors is large (or even larger than the number of observations), it becomes difficult to build stable and interpretable regression models. Many predictors may be correlated, irrelevant, or simply add noise to the model — increasing the risk of overfitting.

Instead of selecting or regularizing individual predictors, another approach is to first reduce the dimensionality of the input space. This is done by creating new features that are combinations of the original ones — capturing most of the variance or predictive information. Two such powerful techniques are:

- **Principal Component Regression (PCR)**
- **Partial Least Squares (PLS)**

Both methods create new uncorrelated variables (called components), and then perform regression using those instead of the original predictors.

Principal Component Regression (PCR)

PCR is a two-step procedure:

1. First, apply **Principal Component Analysis (PCA)** to the standardized predictors $X_1, X_2, \ldots, X_p$ to obtain a set of new variables (called **principal components**), denoted by $Z_1, Z_2, \ldots, Z_p$. These components are linear combinations of the original features:

$$Z_m = \phi_{1m} X_1 + \phi_{2m} X_2 + \cdots + \phi_{pm} X_p$$

   where ϕ_{jm} are the **loadings** of the m-th principal component.

2. Next, use only the first M components $Z_1, Z_2, \ldots, Z_M$ (where $M < p$) to fit a linear regression model to predict the response Y:

$$Y = \theta_0 + \theta_1 Z_1 + \theta_2 Z_2 + \cdots + \theta_M Z_M + \epsilon$$

The idea is that the first few principal components capture most of the variability in the predictors, so we can discard the rest without losing much information.

Standardization: Before applying PCA, we standardize each predictor to have zero mean and unit variance, so that they contribute equally.

How to Choose M? The number of components M is selected using cross-validation — we pick the number that minimizes the prediction error on unseen data.

Prediction: To make predictions:

- Standardize the new input data,
- Project onto the first M principal components,
- Use the fitted regression model.

Partial Least Squares (PLS)

PLS is similar to PCR but with one important difference: while PCR finds components that explain the maximum variance in predictors, **PLS tries to find components that also have high correlation with the response variable Y**.

In other words:

- PCR ignores Y when computing components.
- PLS considers both X and Y, so it often leads to better predictive performance.

PLS constructs components sequentially, each one being a linear combination of the original predictors, but chosen to maximize the covariance with the response.

The procedure involves:

1. Standardizing the predictors.

2. Iteratively constructing new components by combining predictors and ensuring that each captures the most relevant information for predicting $\boldsymbol{Y}$.
3. Using those components in a linear regression model.

Advantages of PCR and PLS:

- Useful when predictors are highly correlated.
- Effective when $\boldsymbol{p > n}$ (more variables than observations).
- Can improve prediction accuracy and model stability.
- Components are uncorrelated — avoids multicollinearity.

Disadvantages:

- PCR components may not be predictive of $\boldsymbol{Y}$ — it only looks at $\boldsymbol{X}$.
- Interpretation becomes harder since components are combinations of features.
- Choosing the number of components requires cross-validation.
- PLS is more complex to implement than PCR.

5.4 Model Evaluation

After building multiple models using different feature selection or regularization techniques, one of the most important steps is to evaluate and compare their performance. The goal is to select the **best-performing model** that generalizes well on unseen data — not just one that performs well on the training set.

In this section, I studied different statistical metrics that help in model evaluation and selection. These metrics provide a trade-off between how well the model fits the training data and its complexity (number of predictors). Overfitting is avoided by penalizing more complex models.

1. C_p Statistic (Mallow's C_p)

$\boldsymbol{C_p}$ is a model selection criterion that estimates the test error by adjusting the training error for the number of predictors used. It is calculated as:

$$\boldsymbol{C_p = \frac{1}{n}\left(\text{RSS} + 2d\hat{\sigma}^2\right)}$$

Where:

- $\boldsymbol{n}$ is the number of observations
- $\boldsymbol{d}$ is the number of predictors in the model (excluding intercept)
- $\boldsymbol{\hat{\sigma}^2}$ is an estimate of the variance of the error terms

Interpretation: Lower $\boldsymbol{C_p}$ values indicate better models. We typically choose the model with the smallest $\boldsymbol{C_p}$, or the one where $\boldsymbol{C_p \approx d}$ (close to number of predictors).

2. Akaike Information Criterion (AIC)

AIC is a general-purpose metric for model selection based on information theory. It balances the model's goodness of fit with its complexity.

$$AIC = -2 \cdot \log(L) + 2d$$

Where:

- L is the likelihood of the model
- d is the number of estimated parameters (including intercept)

Interpretation: Lower AIC implies a better model. Among a set of models, the one with the lowest AIC is preferred.

3. Bayesian Information Criterion (BIC)

BIC is similar to AIC but includes a heavier penalty for the number of predictors:

$$BIC = -2 \cdot \log(L) + d \cdot \log(n)$$

Difference from AIC: Since $\log(n) > 2$ for $n > 7$, BIC imposes a larger penalty than AIC. So it tends to choose simpler models.

Interpretation: Choose the model with the lowest BIC. BIC is more conservative than AIC, favoring smaller models when the dataset is large.

4. Adjusted R^2

While R^2 measures how well a model explains the variability in the data, it always increases when more predictors are added — even if they're irrelevant. To fix this, we use the **Adjusted R^2**:

$$\text{Adjusted } R^2 = 1 - \frac{(1 - R^2)(n - 1)}{n - d - 1}$$

Where:

- R^2 is the standard coefficient of determination
- n is the number of observations
- d is the number of predictors

Interpretation: Unlike R^2, adjusted R^2 can decrease when unnecessary variables are added. So, the model with the **highest Adjusted R^2** is typically considered better.

How I Used These in Practice:

In this internship, I built several linear models using best subset, stepwise, Ridge, Lasso, and Elastic Net. I used these evaluation metrics to compare them and select the best-performing one. For example:

- In subset selection methods, I used C_p, AIC, and BIC to choose the optimal number of predictors.
- For Ridge and Lasso, I used cross-validation to choose λ, and then evaluated models using Adjusted R^2 and validation error.
- I observed how AIC and BIC lead to simpler models than C_p when dataset size is large.

Each metric provides a unique perspective. Therefore, using a combination of these helps make a more informed and robust model selection decision.

After studying and implementing various types of linear models — from basic regression to regularized and dimension-reduced techniques — I realized that linear models, despite their simplicity and interpretability, are limited in capturing complex relationships when the data is highly non-linear.

In many real-world problems, the true relationship between predictors and the response is not strictly linear. This is where **non-linear models (eg.. tree based models)** become important, as they provide the flexibility to capture more intricate patterns in the data without sacrificing predictive power.

In the next chapter, I explore different non-linear modeling techniques that go beyond linear assumptions and help improve model accuracy in such scenarios.

6. Tree-Based Methods

In earlier chapters, I worked with linear models that assume a linear relationship between predictors and the response variable. While these models are simple and interpretable, they often fail when the relationship is highly non-linear or involves complex interactions among variables.

To handle such situations, we need methods that are more flexible and can automatically capture non-linear structures and variable interactions without requiring manual transformations or feature engineering. This is where **tree-based methods** come into play.

Tree-based models work by recursively partitioning the feature space into smaller, more manageable regions, and then making predictions within each region. These models are easy to interpret, can handle both quantitative and qualitative data, and often perform well even with minimal preprocessing.

Throughout this chapter, I explore different types of tree-based methods starting with **Decision Trees**, followed by more advanced techniques such as **Bagging**, **Random Forests**, **Boosting**, and **XGBoost**. Each method builds on the basic decision tree framework and enhances prediction accuracy, stability, and robustness.

In the next section, I begin with decision trees and explain how they work for both regression and classification problems.

6.1 Decision Trees

A **decision tree** is a simple yet powerful model used for both regression and classification problems. The core idea is to split the predictor space into a set of distinct and non-overlapping regions based on certain decision rules. These rules are learned from the data by recursively dividing it into smaller subsets where the target variable becomes more homogeneous.

In regression problems, the tree predicts a quantitative output by taking the average of observations in each terminal node. In classification, it assigns the majority class of each region to new observations.

What makes decision trees important is their **interpretability**, **flexibility**, and **ability to handle both numerical and categorical data** without needing feature scaling or complex transformations. They also naturally model non-linear relationships and interactions between variables.

During my internship, I found decision trees particularly helpful as a foundation to learn more advanced ensemble methods like **Bagging**, **Random Forests**, and **Boosting**, which build on the same core principles but improve stability and accuracy.

In the following sections, I explore how decision trees work for both regression and classification problems in more detail.

6.1.1 Regression Trees

A **regression tree** is a type of decision tree used when the response variable is **quantitative**. It works by dividing the feature space into distinct regions and assigning a constant prediction (typically the average response) for each region. These splits are determined by recursive

binary splitting, where at each step the algorithm looks for the best feature and best threshold to minimize the prediction error.

How It Works:

To build a regression tree, the algorithm performs the following:

- Start with all the training data in a single region.
- At each step, split the data into two regions based on a single predictor X_j and a cut-point s such that:
$$R_1(j,s) = \{X \mid X_j < s\}, \quad R_2(j,s) = \{X \mid X_j \geq s\}$$
- For every possible pair (j,s), we compute the **Residual Sum of Squares (RSS)**:
$$\text{RSS} = \sum_{i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$
where $\hat{y}_{R_1}$ and $\hat{y}_{R_2}$ are the mean responses in those regions.
- Choose the (j,s) pair that gives the lowest RSS, and split accordingly.
- Repeat the process recursively until a stopping criterion is met (e.g., minimum node size or max depth).

Each terminal node (leaf) represents a region where all observations have the same predicted output: the average of the responses in that region.

Why Do We Prune?

Fully grown trees often **overfit** the training data — they capture noise instead of the true underlying pattern. To avoid this, we apply **tree pruning**, which simplifies the model by cutting back branches that do not provide significant improvement in prediction accuracy.

Cost Complexity Pruning (a.k.a. weakest link pruning) is used:

$$C_\alpha(T) = \sum_{m=1}^{|T|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

where $|T|$ is the number of terminal nodes in tree T, and α is a tuning parameter that controls the penalty for complexity. A higher α results in smaller trees.

Selecting Hyperparameters:

To build an effective regression tree, we need to tune several hyperparameters:

- Maximum depth of the tree
- Minimum number of samples required to split a node
- Minimum number of observations in a leaf node
- Complexity parameter α for pruning

These are often chosen using **cross-validation** to find the best trade-off between bias and variance.

Making Predictions:

Once the tree is built (and optionally pruned), to predict for a new observation, we simply:

- Drop the observation down the tree based on the decision rules at each node
- When it reaches a leaf node, return the mean of the training responses in that region

Advantages of Regression Trees:

- Very easy to interpret and visualize
- Automatically handle both numerical and categorical variables
- No need for feature scaling or linear assumptions
- Naturally capture interactions and non-linearities

Disadvantages of Regression Trees:

- They are highly unstable — small changes in data can produce different trees
- Tend to overfit if not pruned properly
- Predictions are not smooth, but piecewise constant
- Generally have lower predictive accuracy compared to ensemble methods

6.1.2 Classification Trees

A **classification tree** is a supervised learning algorithm used when the response variable is **categorical**, typically binary (e.g., Yes/No, 0/1). Unlike regression trees, which predict a continuous outcome, classification trees assign each observation to a class label based on the input features. They split the feature space into regions such that observations in the same region mostly belong to the same class.

Why Classification Trees?

- Many practical problems involve categorical outcomes, and classification trees are naturally designed for such tasks.
- They are highly interpretable and work well even with non-linear decision boundaries.
- They don't require feature scaling and can handle both numerical and categorical inputs.

How the Tree is Built:

The goal of building a classification tree is to divide the predictor space into regions that are as **pure** as possible with respect to the target classes.

At each step, the algorithm chooses:

- A feature X_j and a threshold s

- That best splits the data to reduce **impurity** in the resulting child nodes

Impurity Measures: Gini Index and Entropy

The two most commonly used impurity metrics for classification trees are the **Gini Index** and **Entropy**. Both measure how mixed the classes are within a node.

Gini Index: Gini is the default metric used in many implementations (like scikit-learn). It measures the probability of misclassifying a randomly chosen observation, assuming that it was labeled according to the class distribution in that node.

For a node m, let $\hat{p}_{mk}$ be the proportion of class k observations in that node. The Gini index is defined as:

$$G_m = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) = 1 - \sum_{k=1}^{K} \hat{p}_{mk}^2$$

- If a node contains only one class, Gini = 0 (pure node).
- If the classes are equally mixed (e.g., 50
- Lower Gini means better purity.

Entropy: Another measure based on information theory is the entropy, defined as:

$$D_m = -\sum_{k=1}^{K} \hat{p}_{mk} \log_2(\hat{p}_{mk})$$

- Like Gini, entropy is 0 when the node is pure.
- Entropy grows as the class distribution becomes more mixed.

Comparison: Both Gini and entropy lead to similar trees. Gini tends to be slightly faster to compute, while entropy is more sensitive to class imbalance. In practice, either can be used depending on the application or personal preference.

Tree Growing Process:

- Start with the full training set at the root node.
- Recursively split the data to minimize impurity (Gini or Entropy).
- Continue until a stopping condition is reached (e.g., max depth, min samples in a node).

Pruning the Tree:

A fully grown tree can easily overfit the training data. To control this, we use **Cost Complexity Pruning**, which balances the tree's accuracy and complexity.

$$C_\alpha(T) = \sum_{m=1}^{|T|} Q_m(T) + \alpha|T|$$

- $|T|$ is the number of terminal nodes (leaves)
- $Q_m(T)$ is the impurity at node m

- α is a tuning parameter that penalizes larger trees

We perform cross-validation to select the best α and prune the tree accordingly.

Making Predictions:

- New observations are passed down the tree following learned rules.
- At the terminal node, the predicted class is the majority class.
- We can also use class probabilities from the node if needed.

Tuning Hyperparameters:

Important parameters that control model complexity include:

- `max_depth` – Maximum depth of the tree
- `min_samples_leaf` – Minimum number of samples in a leaf
- `min_samples_split` – Minimum samples needed to perform a split
- `ccp_alpha` – Post-pruning complexity parameter

All these are tuned using cross-validation to achieve a good balance between bias and variance.

Advantages of Classification Trees:

- Easy to interpret and visualize
- No assumptions about feature distribution
- Naturally handles both numerical and categorical variables
- Captures feature interactions automatically

Disadvantages:

- Highly sensitive to small changes in data
- Can overfit without pruning
- Axis-parallel splits may not capture complex decision boundaries

6.2 Ensemble Methods

While individual models like decision trees are easy to understand and implement, they often suffer from high variance (when the tree is deep) or high bias (when the tree is shallow). A single tree may not always give good predictive performance, especially when the dataset is noisy or complex. To overcome these limitations, we use **Ensemble Methods**, which combine the predictions of multiple models to produce a more accurate and stable final output.

The main idea behind ensemble methods is that a group of weak learners, when combined properly, can form a strong learner. By averaging or voting across models, ensemble methods reduce variance and/or bias, leading to better generalization on unseen data.

In this section, I studied and implemented popular ensemble methods such as:

- **Bagging (Bootstrap Aggregating)**

- **Random Forests**
- **Boosting (Gradient Boosting, XGBoost)**

These techniques helped me understand how combining multiple decision trees with different training subsets or boosting strategies can significantly improve accuracy compared to a single tree.

6.2.1 Bagging (Bootstrap Aggregating)

When we use a single decision tree on a training dataset, it tends to have high variance — meaning small changes in the data can lead to very different trees. This makes the model unstable and less reliable on new unseen data. To address this issue, we use a powerful ensemble technique known as **Bagging**, short for **Bootstrap Aggregating**.

The main idea behind bagging is to reduce variance by averaging multiple noisy but approximately unbiased models. Instead of building just one tree, bagging builds many decision trees on different bootstrap samples of the training data, and then averages their predictions (in regression) or takes a majority vote (in classification).

The Bagging Process:

1. Take multiple bootstrap samples from the original training dataset. Each sample is created by sampling with replacement.
2. Train a separate decision tree on each bootstrap sample. These trees are usually grown deep (unpruned), to have low bias.
3. For prediction:
    - In **regression**, average the predictions of all trees.
    - In **classification**, use majority voting to select the final class.

Since each tree is trained on a different subset of data, they make different errors. When we average their outputs, the variance gets reduced, and we get a more stable and accurate final model.

Out-of-Bag (OOB) Error:

An interesting property of bagging is that we can estimate the prediction error without needing a separate validation set. On average, each bootstrap sample leaves out about 1/3 of the training data. These left-out points are called **out-of-bag observations**. We can use them to compute the prediction error by testing each tree on its corresponding OOB points and averaging the result. This gives a reliable estimate of model performance.

Advantages of Bagging:

- Reduces model variance significantly.
- Handles overfitting better than a single decision tree.
- Works well even when the base model is unstable (like deep trees).
- Provides an OOB estimate, saving the need for a separate test set.

Disadvantages of Bagging:

- Loses interpretability — the final model is a black box with many trees.
- Computationally more expensive due to training multiple models.
- Doesn't help much if the base model has high bias (e.g., shallow trees).

6.2.2 Random Forest

Random Forest is one of the most powerful and widely used ensemble learning methods in machine learning. It builds on the idea of **bagging** and improves it by introducing an additional layer of randomness in the model building process. This randomness helps in further reducing the correlation between individual trees, which leads to a more accurate and stable model.

In bagging, we reduce variance by averaging multiple decision trees built on bootstrap samples. However, if many of these trees are very similar (because the same strong predictor dominates all trees), the averaging may not be very effective. To overcome this, random forest adds a twist: instead of considering all features while splitting a node, it randomly selects a subset of features at each split.

How Random Forest Works:

1. Create multiple bootstrap samples from the training data.
2. For each bootstrap sample, build a decision tree:
    - At each split in the tree, instead of considering all $\boldsymbol{p}$ features, select only a random subset of $\boldsymbol{m}$ features (where $\boldsymbol{m < p}$).
    - Choose the best split only among those $\boldsymbol{m}$ features.
3. Grow each tree to full depth (no pruning).
4. For prediction:
    - In **regression**, average the predictions from all trees.
    - In **classification**, use majority voting across trees.

The idea of using a random subset of features at each split helps to de-correlate the trees. This is because different trees explore different patterns and interactions in the data. As a result, the combined model becomes more robust and less likely to overfit.

Key Hyperparameter:

- The number of features to consider at each split, typically:
    - $\boldsymbol{\sqrt{p}}$ for classification tasks
    - $\boldsymbol{p/3}$ for regression tasks

Out-of-Bag (OOB) Error in Random Forest:

Like bagging, random forest also provides an out-of-bag estimate of the model error, which is very helpful for evaluating performance without needing a separate validation set.

Advantages of Random Forest:

- Very high prediction accuracy and robustness to overfitting.
- Automatically handles both numerical and categorical data.
- Works well on both regression and classification problems.
- Reduces variance significantly compared to individual trees.
- Provides feature importance scores to interpret the model.

Disadvantages of Random Forest:

- Less interpretable than a single decision tree.
- More computationally expensive — requires training many trees.
- Can still overfit if the number of trees is too small or too large without tuning.

During my internship, I implemented random forest from scratch and applied it to regression and datasets. I compared its performance with a single decision tree and found that random forest consistently outperformed them in terms of accuracy and generalization. I also explored how feature importance scores provided by random forest helped identify the most influential predictors in the dataset.

6.2.3 Boosting

Boosting is another powerful ensemble technique that builds strong predictive models by combining a large number of weak learners — usually decision trees — in a sequential and smart way. Unlike bagging and random forest, which train trees independently, boosting builds each tree **one at a time**, where each new tree tries to correct the errors made by the previous ones.

The key idea behind boosting is to focus on the observations that were previously misclassified or poorly predicted, and give them more attention in the next iteration. This results in a model that gradually improves with each added tree.

How Boosting Works:

1. Start with an initial prediction — often a constant value (like the mean in regression or log odds in classification).
2. Compute the errors (residuals) based on current predictions.
3. Fit a shallow decision tree (usually a small depth like 1–5) to these residuals.
4. Update the model by adding the prediction from this new tree, scaled by a learning rate λ.
5. Repeat the process for a fixed number of trees or until convergence.

In boosting, each tree tries to fix the mistakes of the earlier trees. The trees are usually not grown deeply, which helps control overfitting.

Learning Rate: The learning rate (also called shrinkage), usually denoted by λ, determines how much influence each tree has on the final prediction. A smaller λ means slower learning, but often leads to better generalization. Typical values are between **0.01** and **0.1**.

Important Hyperparameters:

- Number of trees B
- Tree depth (controls complexity of each learner)
- Learning rate λ

Popular Variants of Boosting:

- **AdaBoost**: Focuses on misclassified observations by increasing their weights.
- **Gradient Boosting**: Fits each new tree on the negative gradient of the loss function.
- **XGBoost**: An optimized and regularized version of gradient boosting, very popular in practice.

Advantages of Boosting:

- Often achieves better accuracy than bagging and random forest.
- Reduces both bias and variance.
- Works well with structured/tabular datasets.
- Handles both regression and classification problems effectively.

Disadvantages of Boosting:

- More sensitive to noisy data and outliers.
- Can overfit if the number of trees is too large or if learning rate is not tuned properly.
- Computationally more expensive due to sequential nature.

6.2.4 XGBoost (Extreme Gradient Boosting)

XGBoost stands for **Extreme Gradient Boosting**. It is an advanced implementation of the gradient boosting algorithm that has become one of the most widely used machine learning techniques in data science competitions and real-world projects. It improves upon traditional boosting methods by focusing on both speed and performance.

What makes XGBoost special is its ability to handle large-scale datasets efficiently, its regularization features to prevent overfitting, and its use of parallel computation and hardware optimization.

Why XGBoost?

- Traditional gradient boosting can be slow and prone to overfitting.
- XGBoost introduces several enhancements to make training faster and more accurate.

- It supports regularization, missing value handling, and tree pruning out of the box.

How XGBoost Works: XGBoost builds an ensemble of trees sequentially like gradient boosting, but uses second-order derivatives (Hessian) of the loss function for more precise updates. It also applies regularization to control the complexity of each tree and prevent overfitting.

For a given iteration, the objective function to minimize is:

$$\text{Obj}(t) = \sum_{i=1}^{n} \ell(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Where:

- ℓ is the loss function (e.g., squared loss or log loss),
- f_t is the new function (tree) added at step t,
- $\Omega(f) = \gamma T + \frac{1}{2}\lambda \sum w_j^2$ is the regularization term penalizing the number of leaves T and the leaf weights w_j.

Using second-order Taylor approximation, it calculates optimal tree splits based on gradients and Hessians, leading to faster convergence and more stable learning.

Important Features of XGBoost:

- **Regularization (L1 and L2):** Helps prevent overfitting.
- **Sparsity-aware Split Finding:** Handles missing values automatically.
- **Tree Pruning:** Uses a max depth or gain threshold to prune trees smartly.
- **Column Subsampling:** Similar to random forest, it samples features at each split.
- **Parallelization:** Trains trees using multiple cores, speeding up the process.

Hyperparameters in XGBoost: Some key hyperparameters include:

- `n_estimators` – Number of boosting rounds (trees)
- `learning_rate` – Shrinkage factor η
- `max_depth` – Maximum tree depth
- `subsample` – Row subsampling ratio
- `colsample_bytree` – Column sampling by tree
- `reg_alpha, reg_lambda` – L1 and L2 regularization

Advantages of XGBoost:

- Excellent prediction performance in both regression and classification.
- Built-in regularization reduces overfitting.
- Efficient handling of missing values.
- Fast training using parallel processing and optimization techniques.

- Highly tunable and scalable to large datasets.

Disadvantages of XGBoost:

- Requires careful hyperparameter tuning.
- Computationally heavier than simpler models.
- Less interpretable than single decision trees.

During my internship, I used XGBoost on multiple real-world datasets and compared its performance with traditional decision trees and random forests. I found that when hyperparameters were well-tuned using techniques like grid search and cross-validation, XGBoost consistently outperformed other models in both accuracy and generalization.

Conclusion:

In this chapter, I explored various tree-based methods including decision trees, bagging, random forest, boosting, and XGBoost. Among all these techniques, I found that **XGBoost consistently delivered the best performance** in terms of both accuracy and generalization across different datasets. Its ability to handle missing values, incorporate regularization, and utilize advanced optimization strategies made it stand out as a powerful and reliable model for both regression and classification tasks.

7. Probabilistic Modeling

In most of the previous chapters, I focused on predictive models that directly output either a continuous value or a classification label. But in many real-world problems, just giving a prediction is not enough — we also want to know how **certain or uncertain** that prediction is.

Probabilistic modeling comes into the picture when we want to associate a degree of confidence or probability with our predictions. Instead of giving just a fixed output, we model the **entire distribution of the output variable**, which allows us to make more informed decisions. For example, in a medical diagnosis system, it's not just important to predict whether a disease is present, but also to understand how confident the model is in that prediction. A 95% confidence might lead to a different action than a 55% confidence.

In daily life, uncertainty is everywhere. If we forecast rain, we also want to know how likely it is — is it a 20% chance or 90%? Probabilistic modeling helps us answer these types of questions in a formal and structured way.

During my internship, I explored how probabilistic models can quantify uncertainty and provide full predictive distributions instead of just point estimates. This gives deeper insight into the reliability of a model's output and allows us to reason under uncertainty — which is crucial in fields like healthcare, finance, and risk management.

In this chapter, I present the core concepts and mathematical foundations of probabilistic modeling, followed by practical methods like **Bayesian Linear and Logistic Regression**, using techniques such as **Maximum Likelihood Estimation (MLE)**, **Variational Inference (VI)**, **Markov Chain Monte Carlo (MCMC)**, and **Laplace Approximation**.

7.1 Bayesian Modeling

In real-world scenarios, we rarely make decisions based on fixed answers — instead, we deal with **uncertainty**, and we continuously update our beliefs as we receive new information. This is exactly what **Bayesian Modeling** helps us do.

At its core, Bayesian modeling is a statistical approach that allows us to update our predictions or beliefs about a situation using **Bayes' Theorem**. It is built upon the idea that probability represents a degree of belief or uncertainty about a particular event, and this belief can be revised as we observe more data.

Why Bayes' Rule?

Bayes' theorem gives us a formal way to combine prior knowledge with observed data. It tells us how to go from what we already believe (prior) and the likelihood of observed data to what we should believe now (posterior). This is written as:

$$P(\theta \mid X) = \frac{P(X \mid \theta) \cdot P(\theta)}{P(X)}$$

Here:

- $P(\theta)$ is the **prior** — our belief about the parameter θ before seeing the data.
- $P(X \mid \theta)$ is the **likelihood** — how likely the observed data X is, given the parameter.

- $P(X)$ is the **evidence** or marginal likelihood.
- $P(\theta \mid X)$ is the **posterior** — updated belief after observing data.

This process of updating beliefs is how we learn in everyday life. For example, if I believe it's going to rain (prior) and I see dark clouds (data), my belief becomes stronger (posterior). Bayesian modeling formalizes this idea in mathematical terms.

Why We Need Central Limit Theorem (CLT)

Before we dive deeper into Bayesian methods, it's important to understand the **Central Limit Theorem**, which is a key foundation for many probabilistic approaches.

The CLT states that the sampling distribution of the mean of a large number of independent, identically distributed random variables tends to follow a **normal (Gaussian) distribution**, regardless of the original distribution. This is why the normal distribution shows up so frequently in statistics — it allows us to model uncertainty about estimates like means and coefficients using a bell-shaped curve.

This justifies why we often assume a normal distribution in Bayesian modeling — especially when working with continuous outcomes — and it makes computations easier and more interpretable.

Normal (Gaussian) Distribution

The normal distribution is a continuous probability distribution defined by two parameters: mean (μ) and variance (σ^2). Its probability density function is:

$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

In Bayesian inference, we often assume the prior and/or likelihood follow a normal distribution because of its mathematical convenience and theoretical justification from CLT.

How Bayesian Modeling is Different from Classical Probabilistic Modeling

Both probabilistic and Bayesian models aim to capture uncertainty, but they approach it differently:

- **Probabilistic Modeling** typically focuses on estimating the probability of the output given input data, using fixed model parameters. The uncertainty comes mainly from the data.
- **Bayesian Modeling**, on the other hand, treats the model parameters themselves as random variables. It starts with a belief (prior) and updates it using observed data to get a refined belief (posterior).

In simpler terms, probabilistic modeling gives us point predictions and estimates, while Bayesian modeling gives us full distributions — allowing us to quantify uncertainty about both the output and the parameters of the model.

7.1.1 Bayesian Linear Regression

In traditional (frequentist) linear regression, we estimate a fixed set of coefficients $\boldsymbol{\beta}$ that minimize the residual sum of squares between the observed responses and predicted values. But in many real-life scenarios, these estimates are uncertain, and we want to express that uncertainty. This is where **Bayesian Linear Regression (BLR)** comes in.

In Bayesian Linear Regression, instead of finding a single "best" value for the parameters, we treat the coefficients themselves as random variables and estimate their **posterior distribution** based on the observed data. This allows us to not only make predictions but also quantify how confident we are about those predictions.

Bayesian Linear Model Setup:

Let us consider the standard linear regression model:

$$\boldsymbol{y = X\beta + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2 I)}$$

Here:

- $\boldsymbol{y}$ is the $\boldsymbol{n \times 1}$ response vector
- $\boldsymbol{X}$ is the $\boldsymbol{n \times p}$ predictor matrix
- $\boldsymbol{\beta}$ is the $\boldsymbol{p \times 1}$ vector of coefficients
- $\boldsymbol{\epsilon}$ is the Gaussian noise

In Bayesian regression, we place a prior distribution on $\boldsymbol{\beta}$:

$$\boldsymbol{\beta \sim \mathcal{N}(\mu_0, \Sigma_0)}$$

This expresses our belief about the coefficients before observing the data.

Posterior Distribution:

Using Bayes' theorem, we compute the posterior distribution over the coefficients:

$$\boldsymbol{p(\beta \mid X, y) \propto p(y \mid X, \beta) \cdot p(\beta)}$$

Where:

- $\boldsymbol{p(\beta)}$ is the prior
- $\boldsymbol{p(y \mid X, \beta)}$ is the likelihood (based on the Gaussian noise model)
- $\boldsymbol{p(\beta \mid X, y)}$ is the posterior — our updated belief after observing data

Since both prior and likelihood are Gaussian, the posterior is also Gaussian:

$$\boldsymbol{\beta \mid X, y \sim \mathcal{N}(\mu_n, \Sigma_n)}$$

Where:

$$\boldsymbol{\Sigma_n = \left(\Sigma_0^{-1} + \frac{1}{\sigma^2} X^\top X\right)^{-1}, \quad \mu_n = \Sigma_n \left(\Sigma_0^{-1}\mu_0 + \frac{1}{\sigma^2} X^\top y\right)}$$

This formula gives us the entire distribution over the coefficients, not just a point estimate.

Prediction for New Data:

Given a new data point x^*, the predictive distribution of the output y^* is also Gaussian:

$$p(y^* \mid x^*, X, y) = \mathcal{N}\left(x^{*\top}\mu_n,\ x^{*\top}\Sigma_n x^* + \sigma^2\right)$$

This means we can not only make a prediction $x^{*\top}\mu_n$, but also provide a confidence interval around that prediction — which is something standard regression doesn't offer directly.

Advantages of Bayesian Linear Regression:

- **Uncertainty Quantification:** We get a full distribution over coefficients and predictions.
- **Incorporates Prior Knowledge:** Priors can encode domain expertise or historical data.
- **Regularization Effect:** The prior acts like a regularizer, reducing overfitting — especially when the prior has small variance (shrinkage effect).
- **Robust to Small Datasets:** It performs better when data is limited, due to prior information.

Disadvantages:

- **Computational Complexity:** Requires matrix inversion and integration; not scalable to large datasets.
- **Choice of Prior:** Results can be sensitive to the choice of prior distribution.
- **Interpretability:** Posterior distributions can be harder to interpret than single estimates.

7.1.2 Bayesian Logistic Regression

In traditional logistic regression, we treat the model parameters β as fixed and estimate them using maximum likelihood. However, this approach does not provide uncertainty in our estimates. In many real-world situations, knowing how confident we are in our prediction is as important as the prediction itself. To address this, we use **Bayesian Logistic Regression**, where we place a prior distribution over the parameters and obtain a full posterior distribution after observing data.

Model Definition:

In logistic regression, the probability of the output y_i being 1 given input x_i is modeled as:

$$p(y_i = 1 \mid x_i, \beta) = \sigma(x_i^\top \beta) = \frac{1}{1 + e^{-x_i^\top \beta}}$$

where $\sigma(\cdot)$ is the sigmoid function.

In the Bayesian setting, we place a prior over the weights:

$$\beta \sim \mathcal{N}(0, \sigma^2 I)$$

Using Bayes' rule, the posterior becomes:

$$p(\beta \mid X, y) = \frac{p(y \mid X, \beta) \cdot p(\beta)}{p(y \mid X)}$$

Likelihood Function:

$$p(y \mid X, \beta) = \prod_{i=1}^{n} \sigma(x_i^\top \beta)^{y_i} (1 - \sigma(x_i^\top \beta))^{1-y_i}$$

The prior is Gaussian, but the likelihood is not conjugate to the prior in logistic regression, which makes the posterior **analytically intractable**. That's why we need to approximate it using different techniques.

Before Approximating Posterior: MLE and MAP

- **Maximum Likelihood Estimation (MLE):**

  $$\hat{\beta}_{MLE} = \arg\max_{\beta} \log p(y \mid X, \beta)$$

  This gives a point estimate based only on data, with no prior belief.

- **Maximum A Posteriori (MAP):**

  $$\hat{\beta}_{MAP} = \arg\max_{\beta} \log p(\beta \mid X, y) = \arg\max_{\beta} [\log p(y \mid X, \beta) + \log p(\beta)]$$

  MAP includes both likelihood and prior, but still gives only one value.

Why Posterior is Intractable?

The logistic function inside the likelihood causes the posterior:

$$p(\beta \mid X, y) \propto \prod_{i=1}^{n} \sigma(x_i^\top \beta)^{y_i} (1 - \sigma(x_i^\top \beta))^{1-y_i} \cdot \mathcal{N}(\beta \mid 0, \sigma^2 I)$$

to be non-conjugate. The denominator (marginal likelihood or evidence) cannot be computed in closed form.

So we need to approximate the posterior using methods like:

- Laplace Approximation
- Variational Inference
- Markov Chain Monte Carlo (MCMC)

1. Laplace Approximation

This method approximates the posterior with a Gaussian centered at the MAP estimate.

- Compute the MAP estimate $\hat{\beta}_{MAP}$
- Compute the Hessian $H = -\nabla^2 \log p(\beta \mid X, y)$ at $\hat{\beta}$
- Approximate:

$$p(\beta \mid X, y) \approx \mathcal{N}(\hat{\beta}, H^{-1})$$

Intuition: We are approximating the posterior shape with a normal distribution near its peak.

Limitation: This works well only if the posterior is close to Gaussian.

2. Variational Inference (VI)

This method turns inference into optimization. We choose a simpler distribution $q(\beta)$ and minimize KL divergence:

$$\mathrm{KL}(q(\beta) \| p(\beta \mid X, y))$$

Instead, we maximize the ELBO:

$$\mathrm{ELBO}(q) = \mathbb{E}_{q(\beta)}[\log p(y \mid X, \beta)] - \mathrm{KL}(q(\beta) \| p(\beta))$$

Steps:

1. Choose family $q(\beta)$ (e.g., Gaussian with diagonal covariance)
2. Optimize ELBO using gradient-based methods

Pros: Fast, scalable to large datasets **Cons:** The approximation depends on the chosen $q(\beta)$

3. MCMC (Markov Chain Monte Carlo)

This method generates samples directly from the posterior:

$$\beta^{(1)}, \beta^{(2)}, ..., \beta^{(S)} \sim p(\beta \mid X, y)$$

Metropolis-Hastings:

- Propose new β' using a proposal distribution $q(\beta' \mid \beta^{(t)})$
- Accept with probability:

$$A = \min\left(1, \frac{p(\beta' \mid X, y)}{p(\beta^{(t)} \mid X, y)} \cdot \frac{q(\beta^{(t)} \mid \beta')}{q(\beta' \mid \beta^{(t)})}\right)$$

Hamiltonian Monte Carlo (HMC): More efficient by using gradient information to make better proposals.

Pros: Very accurate **Cons:** Computationally expensive

Prediction on New Data

Once we have the posterior or its approximation, we can compute the predictive probability:

$$p(y^* = 1 \mid x^*) = \int \sigma(x^{*\top}\beta) \cdot p(\beta \mid X, y)\, d\beta$$

This integral is again intractable, but we can approximate using:

- Monte Carlo: Average over samples from MCMC:

$$p(y^* = 1 \mid x^*) \approx \frac{1}{S} \sum_{s=1}^{S} \sigma(x^{*\top}\beta^{(s)})$$

- Use mean from VI or Laplace:

$$p(y^* = 1 \mid x^*) \approx \sigma(x^{*\top}\mathbb{E}_{q(\beta)}[\beta])$$

Intercept and Standardization

- Standardization of inputs is essential before training.
- Intercept is modeled separately or recovered after inverse scaling.

Advantages:

- Gives full uncertainty estimation on parameters
- Better generalization due to prior
- Predicts probabilities with confidence

Disadvantages:

- Posterior is not tractable — needs approximation
- MCMC is slow for large datasets
- VI and Laplace are approximations, may lose accuracy

8. Datasets and Implementation Results

In this chapter, I present the two datasets I used during my internship and describe how I implemented various statistical and machine learning models on them. Each dataset was chosen to explore and apply a range of models — one focused on regression tasks and the other on classification problems.

1. Diamond Price Prediction Dataset (Kaggle):

This dataset is used for a regression problem where the response variable is **price**, a continuous value. It includes features like carat, cut, color, clarity, depth, and other physical attributes of diamonds. On this dataset, I implemented the following regression models:

- Linear Regression
- Ridge Regression
- Lasso Regression
- Regression Tree
- Random Forest
- XGBoost
- Bayesian Linear Regression

The objective was to study how each model performs when predicting a continuous target and how well they generalize using proper evaluation metrics.

2. Heart Disease Dataset:

This dataset is for a **binary classification task**, where the goal is to predict whether a patient is likely to have heart disease (Yes/No). It includes features such as age, cholesterol, resting blood pressure, and other health indicators. I applied the following classification models:

- Logistic Regression
- K-Nearest Neighbors (KNN)
- Bayesian Logistic Regression using:
  - Laplace Approximation
  - Variational Inference
  - MCMC Sampling

These implementations helped me explore how probabilistic and traditional classification models handle real medical data.

Objective of This Chapter:

In this chapter, I applied and compared all the models I studied on two real datasets — one with a continuous response and one with a categorical response. All models were implemented

manually in Python to deepen my understanding of their mathematical foundations and practical behavior.

8.1 Daimond Dataset

8.1.1 Dataset Overview and EDA Results

In this section, I worked with the Diamond Price Prediction dataset from Kaggle. The original dataset contains over 53,000 rows and multiple features such as `carat`, `cut`, `color`, `clarity`, `depth`, `table`, and dimensions `x, y, z`. My objective was to predict the continuous response variable **price**, using various supervised regression models.

Preprocessing and Cleaning:

- Removed irrelevant or less impactful categorical variables like `clarity` based on domain knowledge and correlation analysis.
- Filtered the dataset to retain only `cut = Premium or Ideal` and `color = E or F` to improve interpretability and reduce one-hot encoding complexity.
- Handled missing or invalid data (e.g., rows with `z = 0`) and ensured all numeric columns were clean.
- Performed one-hot encoding to convert categorical variables into numeric format, such as `cut_Premium`, `color_F`.

Final Features Used for Modeling:

- `price` (target), `carat`, `depth`, `table`, `x`, `y`, `z`, `cut_Premium`, `color_F`
- Final dataset shape after preprocessing: **12,397 rows and 9 features**

Top 5 Correlated Predictors with Price: Based on Pearson correlation analysis: `carat`: 0.92, `x`: 0.88,`z`: 0.87,`y`: 0.85, `cut_Premium`: 0.13

Insights from Correlation Matrix: As shown in the heatmap below, `carat`, `x`, `y`, and `z` are highly positively correlated with `price`, making them key predictors. Features like `depth` show weak or negligible correlation.

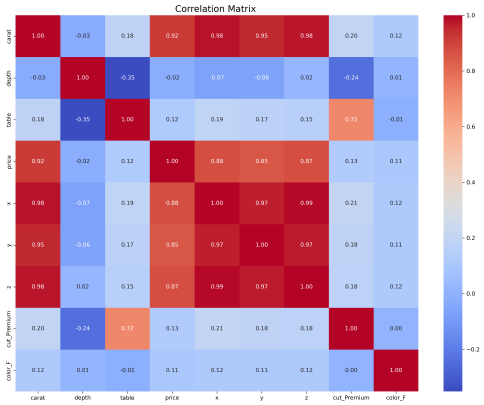


Figure 2: *Correlation Matrix*

This correlation study was important in guiding feature selection and understanding multi-collinearity among predictors before applying any regression techniques.

8.1.2 Results of Models

Linear Regression Results

Estimated Coefficients:
Intercept: -114.4105, **carat**: 8830.5271, **cut_Premium**: -404.2808, **color_F**: -12.7866, **depth**: -17.0198, **table**: -22.9759

Train R^2: 0.84 Test R^2: 0.85

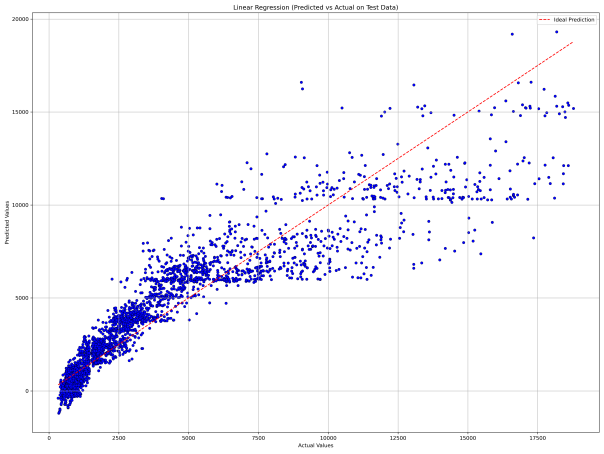


Figure 3: *Predicted vs Actual*

Ridge Regression Results

Best λ: 10.2353
Intercept: 16163.5317, **carat**: 14779.5520, **depth**: -122.0059, **table**: -39.7298, **x**: -2335.4889, **y**: -52.1515, **z**: 44.0182, **cut_Premium**: -334.5713, **color_F**: -0.6881

Train R^2: 0.86 Test R^2: 0.86

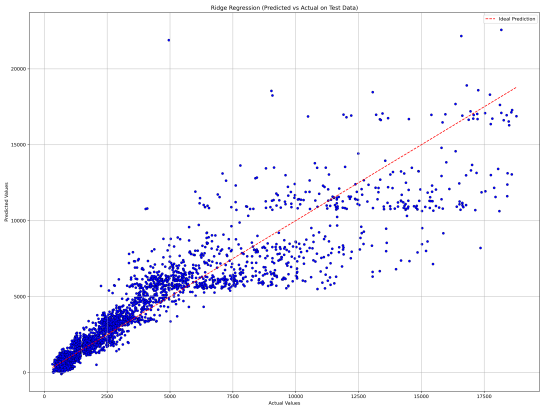


Figure 4: *Predicted vs Actual*

Lasso Regression Results

Best λ: 0.6280
Intercept: 17987.7309, **carat**: 15341.6700, **depth**: -137.8355, **table**: -41.0028, **x**: -3062.7440, **y**: 381.1628, **z**: 176.0801, **cut_Premium**: -312.0506, **color_F**: -0.5024

Train R^2: **0.86** **Test** R^2: **0.86**

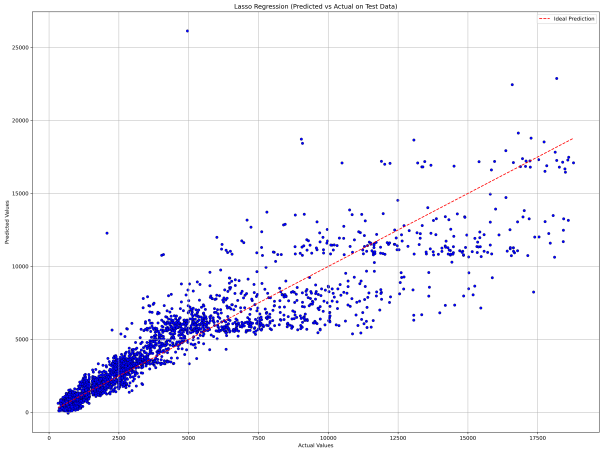


Figure 5: *Predicted vs Actual*

Regression Tree Results

Train R^2: **0.8902** **Test** R^2: **0.8769**

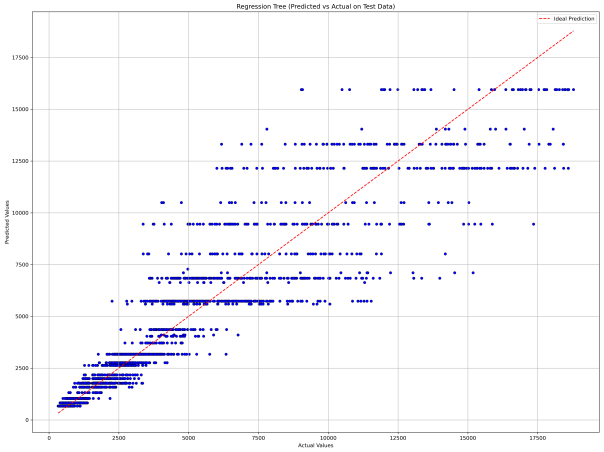


Figure 6: *Predicted vs Actual*

Random Forest Results

Train R^2: **0.8898** **Test** R^2: **0.8817**

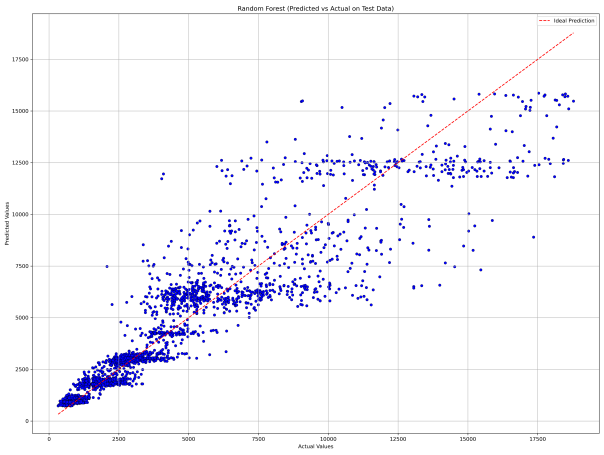


Figure 7: *Predicted vs Actual*

XGBoost Results

Train R^2: 0.9294 Test R^2: 0.8775

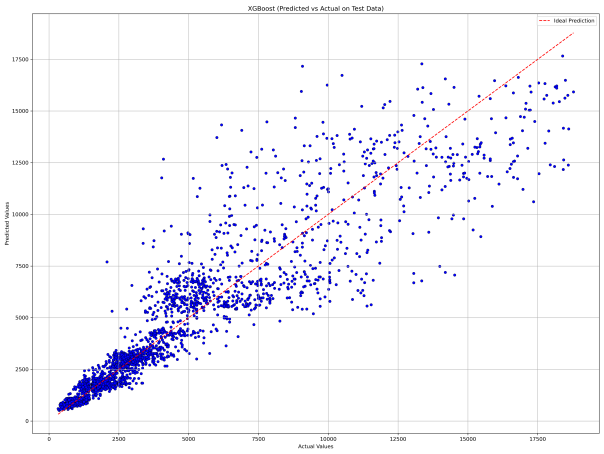


Figure 8: *Predicted vs Actual*

Bayesian Linear Regression Results

Posterior Mean Coefficients (with Uncertainty):
Intercept: 18492.6351 ± 0.0101, **carat**: 15327.7246 ± 0.1425,
depth: -149.4332 ± 0.0164, **table**: -38.5196 ± 0.0080,
x: -3714.5177 ± 0.2282, **y**: 939.6361 ± 0.2280, **z**: 347.5968 ± 0.1948,
cut_Premium: -271.9744 ± 0.0312, **color_F**: 4.2379 ± 0.0202

Train R^2: 0.8627 Test R^2: 0.8251

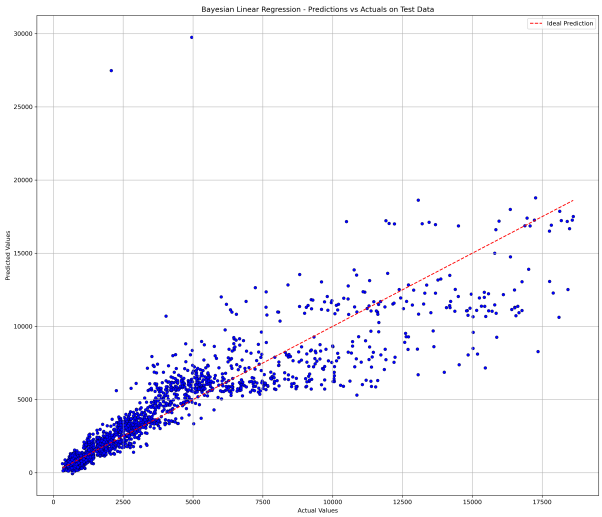


Figure 9: *Predicted vs Actual*

Uncertainty Visualizations (Bayesian Linear Regression)

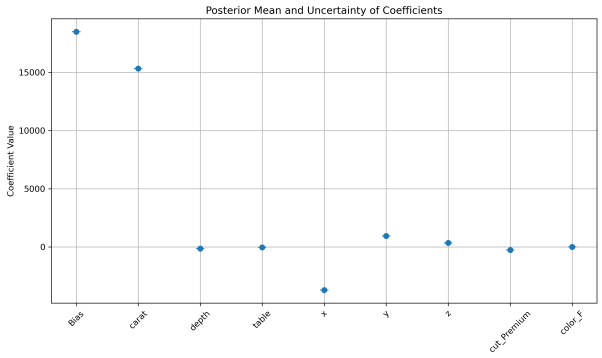


Figure 10: *Posterior Means and Standard Deviations of Coefficients in Bayesian Linear Regression.*

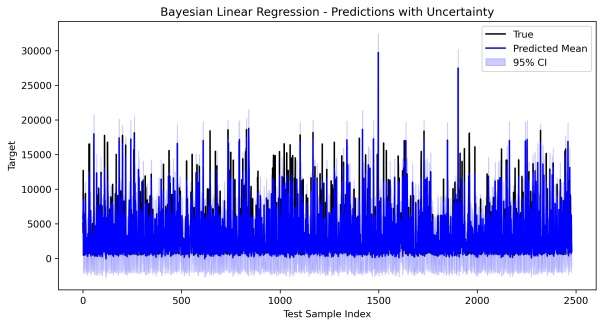


Figure 11: *Bayesian Linear Regression Predictions with 95% Confidence Intervals.*

8.1.3 Model Comparison and Conclusion

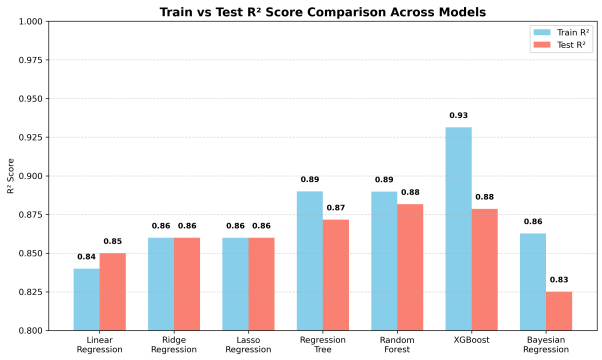


Figure 12: Train vs Test R^2 Score Comparison Across Regression Models

Model-wise Observations:

- **Linear Regression**: Acts as a good baseline with **moderate accuracy** (**Train R^2 = 0.84**, **Test R^2 = 0.85**). However, it assumes strict linearity and lacks flexibility in handling correlated features.
- **Ridge Regression**: Handles **multicollinearity** better than linear regression by adding L_2 regularization. Accuracy slightly improves and coefficients remain interpretable. It reduces overfitting in presence of correlated features.
- **Lasso Regression**: Provides **feature selection** due to L_1 penalty. If **interpretability** is the goal, Lasso is the best among linear models — offering both **high accuracy** (**Test R^2 = 0.86**) and **sparse coefficients**.
- **Regression Tree**: Introduces **non-linearity** and performs better than linear models, but can be slightly unstable and prone to overfitting without pruning. Test R^2 is **0.87**.
- **Random Forest**: An ensemble of trees — reduces overfitting from individual trees. Offers **high accuracy and stability** (**Test R^2 = 0.88**) but is less interpretable.
- **XGBoost**: Achieves the **highest accuracy** among all models (**Train R^2 = 0.93**, **Test R^2 = 0.88**) due to boosting, handling complex patterns very well. Ideal when **prediction performance** is the priority.
- **Bayesian Linear Regression**: Slightly lower test R^2 (**0.83**), but offers **uncertainty estimates** for both predictions and coefficients. Useful when modeling under uncertainty or in scientific applications.

Conclusion: Each model addresses the limitations of the previous one — Ridge improves upon Linear by stabilizing coefficients, Lasso brings interpretability, Tree models capture non-linearity, and Bayesian Regression provides uncertainty. **If accuracy matters most, XGBoost is ideal**; **for interpretability, choose Lasso**; and **for uncertainty-aware applications, prefer Bayesian Linear Regression**.

8.2 Heart Disease

8.2.1 Dataset Overview and EDA Results

In this section, I worked with the Heart Disease dataset from the ISLP library. The dataset contains 303 observations and 14 features, including patient attributes like `Age`, `Sex`, `Cholesterol` (`Chol`), and `Maximum Heart Rate` (`MaxHR`). My objective was to address a binary classification problem: predicting the presence of heart disease, represented by the response variable **AHD** (Absence of Heart Disease), using various supervised classification models.

Preprocessing and Cleaning:

- Handled missing values present in the `Ca` (4 missing) and `Thal` (2 missing) columns by imputing or removing them to ensure a complete dataset.
- Converted the binary target variable `AHD` from object type to numeric, mapping `'No'` to 1 and `'Yes'` to 0 for modeling.
- Performed one-hot encoding on categorical predictors like `ChestPain` and `Thal` to convert them into a numerical format suitable for classification algorithms. This resulted in new features like `ChestPain_nonanginal` and `Thal_normal`.

Final Features Used for Modeling:

- `AHD` (target), and 16 predictor variables including `Age`, `Sex`, `RestBP`, `Chol`, `MaxHR`, `Oldpeak`, and the newly created dummy variables.
- Final dataset shape after preprocessing: **303 rows and 17 features**.

Top 5 Correlated Predictors with Response (AHD):

Based on Pearson correlation analysis (by absolute value):

Thal_normal: 0.52, **Thal_reversable**: -0.48, **Ca**: -0.46, **ExAng**: -0.43, **Oldpeak**: -0.42.

Insights from Correlation Matrix:

As shown in the heatmap below, several features show a significant correlation with the target variable `AHD`. Predictors like `Thal_normal`, `MaxHR`, and certain types of chest pain have a positive correlation with the absence of heart disease. Conversely, features like `Thal_reversable`, `Ca`, `ExAng` (exercise-induced angina), and `Oldpeak` are negatively correlated, indicating their association with the presence of heart disease.

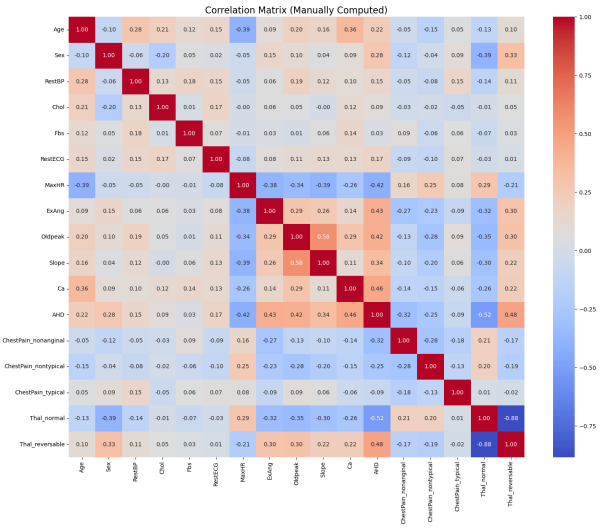


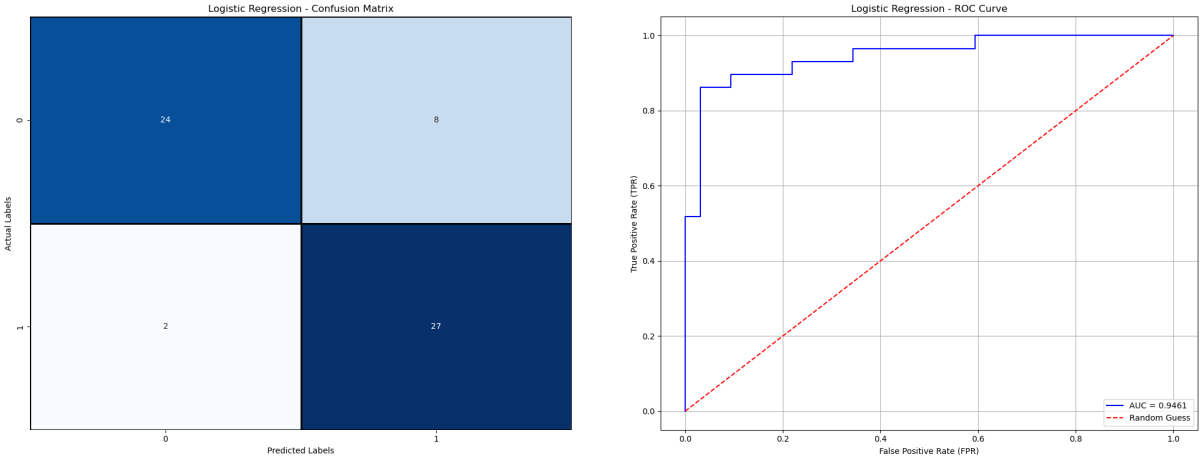
Figure 13: *Correlation Matrix of the Heart Disease Dataset Features*

8.2.2 Results of Models

In this section, I present the results of the classification models applied to the Heart Disease dataset. All models were evaluated on their ability to predict the presence of heart disease (AHD).

Logistic Regression Results

Train Acc: 0.83 **Test Acc:** 0.84 **Precision:** 0.85 **Recall:** 0.84 **F1-Score:** 0.84 **AUC:** 0.91



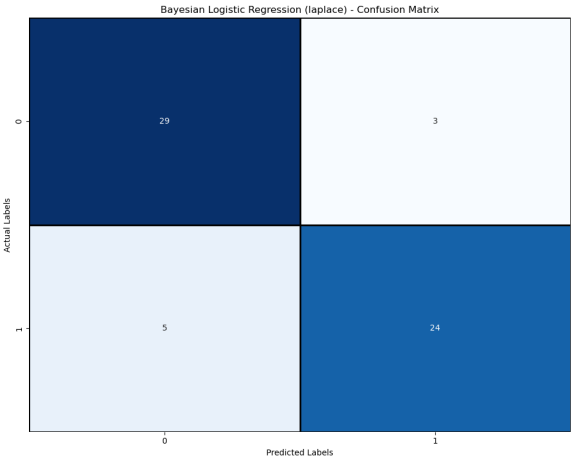
(a) Confusion Matrix (b) ROC Curve

Figure 14: Logistic Regression Performance on Test Data

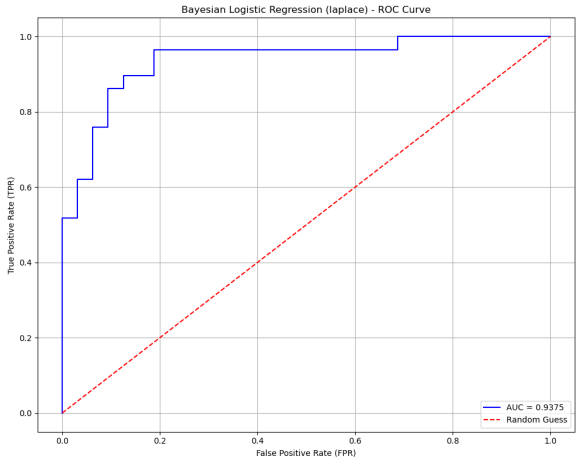
Bayesian Logistic Regression with Laplace Approximation

This model approximates the posterior distribution with a Gaussian, providing both predictions and uncertainty estimates.

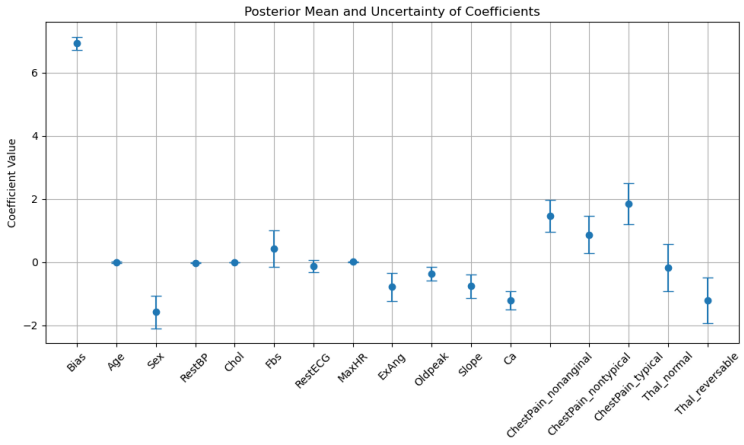
Train Acc: 0.85 **Test Acc:** 0.87 **Precision:** 0.87 **Recall:** 0.87 **F1-Score:** 0.87 **AUC:** 0.93



(a) Confusion Matrix



(b) ROC Curve



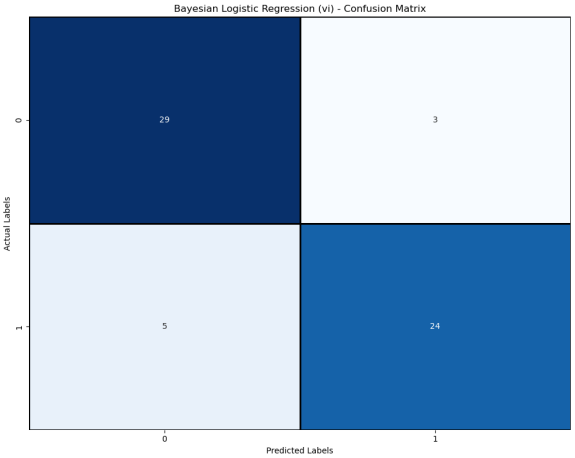
(c) Coefficients with Uncertainty

Figure 15: Bayesian Logistic Regression (Laplace) Performance

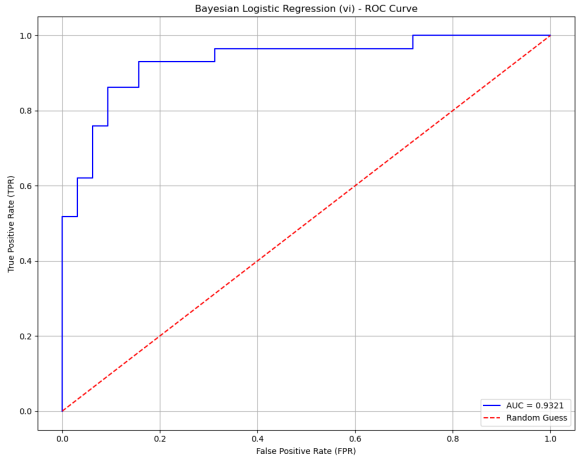
Bayesian Logistic Regression with Variational Inference (VI)

VI offers a faster alternative to MCMC for posterior approximation, balancing speed and accuracy.

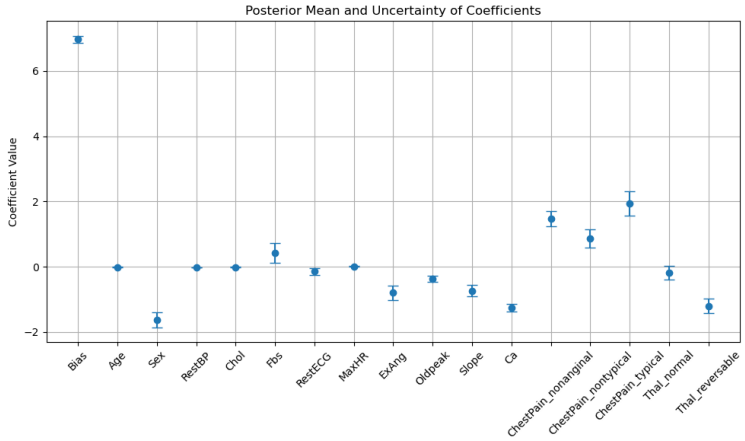
Train Acc: 0.84 **Test Acc:** 0.87 **Precision:** 0.87 **Recall:** 0.87 **F1-Score:** 0.87 **AUC:** 0.93



(a) Confusion Matrix



(b) ROC Curve



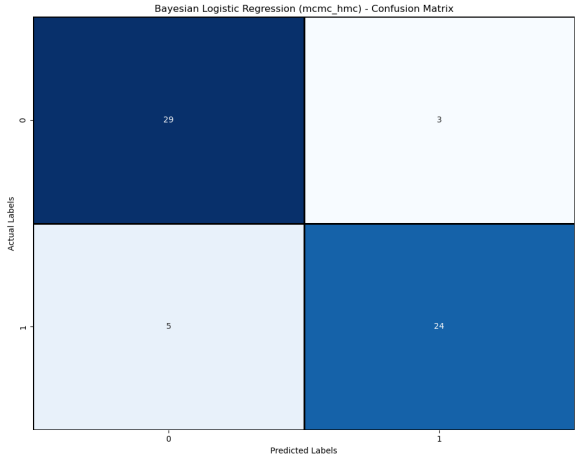
(c) Coefficients with Uncertainty

Figure 16: Bayesian Logistic Regression (VI) Performance

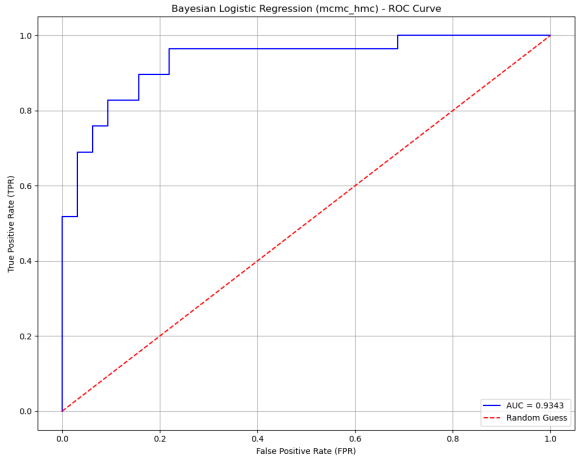
Bayesian Logistic Regression with MCMC

MCMC provides the most accurate posterior estimation by sampling directly from the distribution.

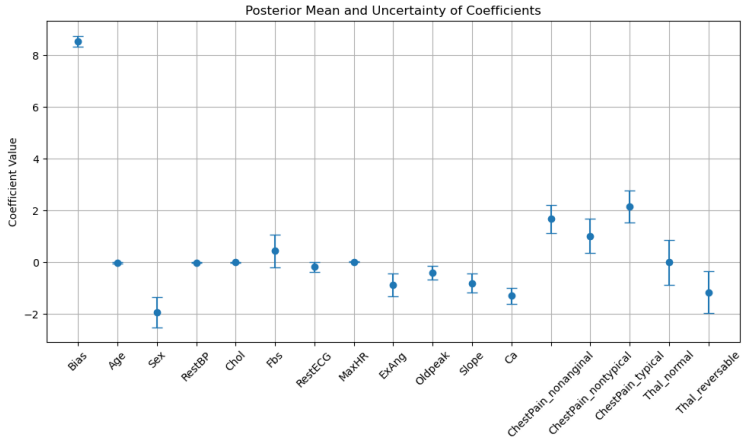
Train Acc: 0.84 **Test Acc:** 0.87 **Precision:** 0.87 **Recall:** 0.87 **F1-Score:** 0.87 **AUC:** 0.93



(a) Confusion Matrix



(b) ROC Curve



(c) Coefficients with Uncertainty

Figure 17: Bayesian Logistic Regression (MCMC) Performance

8.2.3 Model Comparison and Conclusion

Upon evaluating the performance of all classification models on the Heart Disease dataset, it is clear that while all methods performed competently, the Bayesian approaches offered a distinct advantage. The standard Logistic Regression established a strong baseline with a test accuracy of 84% and an AUC of 0.91. However, all three Bayesian Logistic Regression models—using Laplace, VI, and MCMC approximations—consistently outperformed it, achieving a higher test accuracy of 87% and a superior AUC of 0.93. This indicates that for this particular problem, modeling parameter uncertainty provided a more robust and generalizable solution.

Model-wise Analysis:

- **Logistic Regression:** This model served as an excellent, interpretable starting point. Its primary limitation is that it provides single point es

timates for coefficients, offering no insight into the uncertainty of these parameters. It assumes the estimated coefficients are the one ”true” set of values.

- **Bayesian Logistic Regression (Laplace, VI, & MCMC):** These models address the core limitation of the standard approach by treating coefficients as random variables. By incorporating a prior distribution, they introduce a natural form of regularization and, most importantly, yield a full posterior distribution for each parameter. This quantification of uncertainty is likely what contributed to their improved stability and predictive power.
  - The identical top-tier performance across Laplace, VI, and MCMC is noteworthy. It suggests that for this dataset, the posterior distribution was well-behaved enough that even the simpler Gaussian approximation from the **Laplace** method was sufficient to capture the essential information.
  - **Variational Inference (VI)** confirmed these results while providing the tightest uncertainty bounds, demonstrating its efficiency in finding a confident posterior approximation.
  - The fact that the computationally intensive **MCMC**—the most accurate method for posterior sampling—did not yield further gains reinforces the conclusion that the simpler Bayesian methods were perfectly adequate and effective for this task.

Conclusion

For the Heart Disease classification task, the Bayesian logistic regression models were demonstrably superior to the standard frequentist approach. They not only delivered higher predictive accuracy but also provided a crucial layer of insight by quantifying parameter uncertainty. This highlights the practical value of a probabilistic approach in building more reliable and informative clinical prediction models.

9. Conclusion

This internship provided a comprehensive and rigorous exploration of statistical learning, bridging the gap between foundational theory and practical, real-world implementation. The journey began with the core principles of statistical learning, focusing on the fundamental goal of estimating an unknown function f and navigating the critical trade-offs between prediction accuracy and model interpretability, as well as bias and variance. This theoretical grounding was essential for appreciating the nuances of the diverse modeling techniques that followed.

The practical work involved a systematic progression through a hierarchy of models. I began with the implementation of interpretable baselines like Linear and Logistic Regression. Recognizing their limitations, particularly their linear assumptions and vulnerability to overfitting, I moved to advanced techniques such as subset selection and regularization methods like Ridge and Lasso, which offered improved stability and feature selection. To capture the non-linear patterns prevalent in real-world data, I then implemented tree-based methods, progressing from single Decision Trees to powerful ensemble techniques like Random Forest and XGBoost. The final and most advanced phase of the internship involved delving into probabilistic modeling, where I implemented Bayesian Linear and Logistic Regression using sophisticated approximation techniques like Laplace, Variational Inference, and MCMC to quantify model uncertainty.

The application of these models to the Diamond Price Prediction and Heart Disease datasets yielded critical insights. For the regression task on the Diamond dataset, where predictive accuracy was the primary goal, ensemble methods like XGBoost demonstrated superior performance by effectively modeling complex, non-linear relationships. In contrast, for the classification task on the Heart Disease dataset, the Bayesian models proved to be the most effective. They not only achieved the highest predictive accuracy but also provided invaluable uncertainty estimates for their predictions and parameters—a crucial feature for applications in sensitive domains like medical diagnostics.

Ultimately, this internship was not merely an exercise in applying algorithms but a deep dive into their underlying mechanics, made possible by implementing each model from scratch. This hands-on approach solidified my understanding of why certain models are chosen over others and how to navigate their respective strengths and weaknesses. The experience has equipped me with a robust and versatile skill set, enabling me to confidently select, implement, and critically evaluate the appropriate statistical learning tools to solve complex, data-driven problems.

10. References

Core Textbooks

- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning: with Applications in Python*. Springer. This book served as the primary guide for understanding and implementing foundational and advanced supervised learning models, from linear regression to tree-based ensembles.
- Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. This text was instrumental in building a strong theoretical foundation for the probabilistic models explored in the final phase of the internship, particularly for Bayesian linear regression.
- Murphy, K. P. (2023). *Probabilistic Machine Learning: Advanced Topics*. MIT Press. This volume was consulted for its in-depth treatment of advanced inference techniques, providing the mathematical basis for implementing Variational Inference and MCMC for Bayesian logistic regression.

Software and Libraries

- **Python 3.x:** All algorithms and analyses were implemented using the Python programming language, which provided the core environment for this project.
- **NumPy & Pandas:** Utilized extensively for numerical computation, data manipulation, and preprocessing tasks across all datasets.
- **Matplotlib & Seaborn:** Employed for generating all visualizations, including correlation matrices, scatter plots, and performance curves (ROC).
- **Scikit-learn:** Used for data preprocessing utilities, such as data splitting and feature scaling, and for validating the results of my manual model implementations.
- **PyMC3 & ArviZ:** These libraries were used for implementing and diagnosing the Bayesian models, particularly for MCMC sampling.

Datasets

- **ISLP Library:** The source for the Heart Disease dataset used in the classification analysis. The library is a companion to the primary textbook and provided clean, well-structured data for model testing.
- **Kaggle:** The platform from which the Diamond Price Prediction dataset was sourced. Kaggle provided a large, real-world dataset that was ideal for evaluating the performance of various regression models.